

**NEIGHBORHELP: A COMMUNITY-BASED SERVICE FINDER AND
BOOKING PLATFORM**



Presented to the Institute of Computing
Davao del Norte State College Panabo
City, Davao del Norte

In Partial Fulfillment of the
Requirements for the Degree
BACHELOR OF SCIENCE IN INFORMATION TECHNOLOGY

ANILAO, KEAN KYLE F.
GABITO, KENT IVAN MANUEL C.
SEDOLLO, CRISTINE JANE D.

JUNE 2026

CHAPTER 1

INTRODUCTION

Background of the Study

Access to reliable local services has become increasingly important as communities worldwide continue to experience rapid urbanization and digital transformation. Across the globe, the demand for household and personal services such as plumbing, electrical repair, home cleaning, tutoring, and other skilled services has grown alongside increasing population, busy lifestyles, and the expansion of the gig economy. To address this demand, web- and mobile-based service platforms have emerged as effective solutions that connect customers with service providers, offering faster communication, improved accessibility, and more efficient service transactions. Previous studies have shown that cross-platform digital systems enhance service accessibility, operational efficiency, and user convenience while supporting local economic activities and employment opportunities [1],[2],[3].

In the Philippines, the importance of digital technology adoption is reinforced through national initiatives such as the Philippine Digital Strategy and programs of the Department of Information and Communications Technology (DICT), which promote digital inclusion, ICT accessibility, and technology-driven public services [5], [6]. The increasing use of smartphones and internet connectivity further demonstrates the growing reliance on digital platforms for everyday activities and community engagement. Recent statistics from the Philippine Statistics Authority (PSA) revealed that in the Davao Region, 53.4% of individuals aged 10 years and above used the internet in 2024, while 68.1% owned mobile phones, with smartphones serving as the primary device for internet access [7].

In Davao del Norte, the continuous expansion of internet connectivity, public Wi-Fi initiatives, and digital transformation programs has encouraged more residents to use digital platforms for communication, online transactions, business promotion, and accessing local services [8]. Social media platforms, particularly

Facebook groups and community pages, have become common channels for finding individuals who offer short-term or on-demand services such as plumbing, electrical repair, home cleaning, tutoring, and other household services. However, these platforms are primarily designed for social networking rather than service management, making it difficult for users to efficiently search for qualified providers, verify their credibility, compare available services, or manage bookings in an organized manner. As a result, customers may experience inconsistent information, delayed responses, and uncertainty regarding the quality and availability of service providers [9], [10], [11]. Likewise, many local workers and small-scale service providers have limited opportunities to showcase their skills and reach potential customers because there is no centralized platform dedicated to local service discovery and booking. These challenges highlight the need for a community-based digital platform that streamlines service searching, provider verification, communication, and booking while supporting the local economy in Davao del Norte.

Existing studies and service platforms commonly focus on generalized marketplace systems and often lack localized community-centered features [12], [13]. Many available platforms do not fully integrate essential functions such as real-time communication, appointment scheduling, GPS-based provider matching, ratings and reviews, and analytics within a unified system [14]. As a result, local service management remains inefficient and fragmented, limiting user trust, convenience, and overall satisfaction [15].

The proposed NeighborHelp platform supports Sustainable Development Goal (SDG) 8: Decent Work and Economic Growth by expanding employment opportunities for local service providers through digital technology, and SDG 11: Sustainable Cities and Communities by improving community access to essential services through a reliable and organized digital platform [16]. Moreover, the study aligns with the Davao del Norte State College (DNSC) Research, Development, and Extension (RDE) Agenda 2026–2030, particularly under the Innovation and Inclusion (I²) Agenda, which promotes ICT for Development (ICT4D) and the use

of digital technologies to address community needs and foster inclusive local development [17]. It also supports the CHED A.C.H.I.E.V.E. Agenda, specifically the Harmonized SDG-Based Research and Innovation thrust, which encourages higher education institutions to develop research and technological innovations that address societal challenges and contribute to sustainable development, as well as the Integrated Real-Time Data Collection and Analytics enabler, which promotes digital transformation and data-driven service delivery [18]. Furthermore, the study is consistent with the Department of Science and Technology – Harmonized National Research and Development Agenda (DOST-HRNDA) under the Information and Communications Technology (ICT) sector, which prioritizes the development of innovative digital systems and platforms that improve public services, strengthen local economies, and enhance the quality of life through technology-driven solutions [19].

Despite the availability of digital service platforms, there remains a research gap in developing a localized and integrated web and mobile-based community service system tailored to the needs of Philippine communities, particularly in Davao del Norte [20]. Existing platforms primarily focus on isolated functionalities or large-scale marketplace models rather than comprehensive, community oriented solutions [13], [30]. This gap highlights the need for a system that improves accessibility, transparency, and efficiency while supporting local economic participation.

Thus, this study aims to develop and evaluate NeighborHelp, a web and mobile-based community-based platform designed to improve access to local services. The platform will integrate features such as service browsing, in-app communication, appointment scheduling, GPS-based provider matching, ratings and reviews, and data analytics to enhance transparency, service efficiency, and informed decision-making. The findings of this research are expected to contribute to improved local service accessibility, stronger community engagement, and increased economic opportunities within the Philippine context.

Objectives of the Study

The main objective of this project is to design and develop NeighborHelp: A Community-Based Service Finder and Booking Platform that facilitates the discovery, matching, verification, and evaluation of local service providers within the community. Specifically, the project will:

1. To design and implement a GIS-based mapping system for visualizing and locating community service providers based on geographic coordinates, service categories, and service coverage areas
2. To develop and integrate a service provider matching algorithm that recommends qualified service providers according to household service requests, location, availability, verification status, and provider ratings.
3. To integrate a third-party verification agency for validating the identity and qualifications of service providers to enhance platform credibility, trust, and reliability.
4. To apply sentiment analysis techniques to user-generated reviews and feedback for assessing service quality, measuring customer satisfaction, and generating provider reputation indicators.
5. To develop an analytics dashboard that provides real-time insights and visual reports on service requests, provider performance, verification status, user feedback sentiment, booking activities, and community service trends to support informed decision-making and platform management.

Significance of the Study

The proposed Community-Based Service Finder and Booking Platform is expected to provide significant benefits to various stakeholders by improving service accessibility, operational efficiency, and data-driven decision-making.

Community Members. Residents benefit from a reliable and convenient platform for finding nearby and trustworthy service providers. The application reduces the time and effort required to search, communicate, and schedule services while enabling informed decision-making through ratings and reviews.

Local Service Providers. Service providers gain increased visibility within their community and improved access to potential customers. The platform offers an organized channel for showcasing services, managing bookings, and building credibility, which can lead to expanded service opportunities and income growth.

Researchers and Future Students. This study serves as a reference for future research and development of community-based applications. It provides insights into system design, implementation, and evaluation that may guide similar projects in web and mobile service platforms and digital marketplaces.

Scope and Limitations

This study focuses on the design, development, implementation, and evaluation of NeighborHelp, a community-based service finder and booking platform that connects residents with local service providers through a centralized digital system. The platform is designed to support household and community-based services, including home cleaning, plumbing, electrical repair, appliance repair, carpentry, painting, tutoring, babysitting, caregiving, delivery assistance, and other similar on-demand services within Panabo City, Davao del Norte.

NeighborHelp will be developed using a cross-platform framework that supports both web and mobile deployment. Users can access the platform through web browsers on desktop devices or through the dedicated mobile application on Android devices. The responsive design ensures consistent usability and accessibility across different screen sizes and device types. It provides core functionalities such as user registration and authentication, service browsing and searching, appointment booking and scheduling, in-app messaging, GPS-based service provider matching, notifications, and a rating and review system. The

platform employs a weighted scoring algorithm to recommend suitable service providers based on the requested service category, geographic proximity, availability, provider ratings, and booking completion rate. Verification status serves as an eligibility requirement, ensuring that only verified service providers are included in the recommendation process.

To enhance user trust and platform reliability, NeighborHelp incorporates a service provider verification process in partnership with the Public Employment Service Office (PESO) Panabo. This partner organizations are responsible for validating the identity, qualifications, and legitimacy of service providers before they are granted a Verified Service Provider status on the platform. The system also includes basic analytics and sentiment analysis to generate reports on service demand, booking activities, provider performance, verification statistics, and customer feedback, enabling administrators to monitor system usage and support data-driven decision-making.

The initial implementation of NeighborHelp supports community-based service categories with verification requirements based on the nature of the service offered. Technical and regulated services, such as electrical installation, plumbing, computer servicing, carpentry, and air-conditioning repair, require service providers to submit valid competency certificates (e.g., TESDA National Certificates or equivalent credentials) in addition to identity verification through the integrated third-party verification process. Non-regulated community services, such as house cleaning, gardening, delivery assistance, tutoring, babysitting, and caregiving, require identity verification through the integrated third-party verification process, while competency certificates may be optional depending on the selected service category. Before becoming visible to customers, all verified service providers are required to complete their service profiles, which undergo an administrative review to ensure that the submitted information complies with the platform's content standards and service policies. Only approved service profiles are made available for booking. Additional service categories and corresponding verification requirements may be incorporated in future versions of the platform.

The platform facilitates service coordination and supports cashless payment arrangements by displaying service fees and recording transaction information. However, all financial transactions are completed directly between customers and service providers using external payment methods. The system does not process, store, or transfer payments. User information is protected through secure authentication, role-based access control, and privacy measures consistent with the provisions of the Data Privacy Act of 2012 (Republic Act No. 10173).

This study is limited to the implementation and evaluation of the system within Panabo City, Davao del Norte, and only covers household and community-based services. Professional services that require government-issued licenses or specialized professional regulation, such as medical, legal, accounting, and engineering services, are excluded from the platform. The provider recommendation mechanism is limited to a weighted scoring algorithm and does not employ machine learning or advanced artificial intelligence for provider matching. Furthermore, the study does not include integration with online payment gateways, banking institutions, predictive analytics, large-scale commercial deployment, or long-term assessment of user adoption and business sustainability. The study is also limited to the administrative review of provider service profiles based on submitted information and supporting documents. While the platform implements identity verification, profile review, ratings, reviews, and complaint management to reduce risks, it cannot guarantee that verified service providers will not engage in misconduct after approval. Any incidents occurring after service transactions shall be addressed through the platform's reporting, investigation, suspension, and account management mechanisms. These areas are beyond the scope of the present study and may be considered in future enhancements.

Review of Related Literature and Works

Related Literature

The development of digital platforms for service discovery has significantly transformed how individuals access local services. With the rapid growth of web and mobile technologies, service-based applications have become essential tools for connecting customers with providers in a more efficient and organized manner. According to Biørn-Hansen et al., cross-platform applications that integrate both web and mobile functionalities improve accessibility and user experience by enabling seamless interaction across devices [1]. Similarly, other studies on cross platform systems emphasized that integrating mobile and web environments increases usability, availability, and consistency of services across multiple devices [3], [29]. While web-based systems provide broader accessibility, they may suffer from latency issues depending on network conditions, whereas mobile applications offer faster interaction and better engagement through optimized interfaces and offline capabilities [28]. Although cross-platform systems provide significant advantages in flexibility and accessibility, researchers noted that they require more complex development and maintenance processes [29].

One major theme identified in the literature is service matching and recommendation systems. Traditional service discovery methods, such as word of-mouth communication and social media postings, are often unstructured and inefficient. Juho et al. emphasized that these informal approaches create difficulties in verifying service quality, reduce trust, and slow down service transactions [13], [20]. To address these limitations, modern platforms use matching algorithms to connect users with appropriate service providers. Rule-based filtering algorithms commonly match users according to predefined criteria such as category, availability, and location. In contrast, recommendation-based approaches, including collaborative filtering and machine learning algorithms, analyze user behavior and preferences to generate more personalized recommendations [19], [27]. Comparative studies indicate that recommendation-based systems generally

outperform traditional search methods in terms of matching accuracy and user satisfaction because they adapt to user preferences over time [27]. However, these advanced algorithms also require greater computational resources, larger datasets, and more complex data processing compared to simpler rule-based methods [19]. This suggests that while intelligent recommendation systems improve personalization, simpler filtering approaches remain advantageous for systems requiring lower computational costs and easier implementation.

Another important theme in the literature is geolocation and GPS-based services. Location-aware technologies significantly improve the efficiency of service platforms by enabling users to identify nearby providers in real time. Studies revealed that GPS-integrated systems improve response time, matching precision, and overall user satisfaction by prioritizing proximity and provider availability [22], [23]. Compared to non-location-based systems, GPS-enabled platforms reduce search time and enhance service accessibility because users can immediately locate providers within their area [24]. Despite these strengths, researchers also identified limitations associated with real-time location systems, including increased battery consumption, dependency on internet connectivity, and higher processing requirements for continuously updating location data [24]. These findings indicate that geolocation technologies greatly improve operational efficiency but require optimized system design to minimize performance issues.

Booking management and structured transaction systems also emerged as a recurring topic in the reviewed studies. Existing community-based service transactions often rely on informal communication channels such as social media messaging or phone calls, which lack structured scheduling, verification, and accountability mechanisms. Studies on local service-booking platforms found that these limitations frequently lead to delayed responses, inconsistent service quality, and poor coordination between users and providers. Structured booking systems improve efficiency by enabling organized scheduling, automated confirmations, transaction tracking, and centralized communication. Compared to informal systems, platforms with integrated booking management provide more transparent

and reliable service coordination. However, some studies noted that many existing applications implement only partial booking functionalities and fail to integrate them effectively with communication, location, and analytics systems, resulting in fragmented user experiences [16], [17].

Trust, transparency, and provider verification were also highlighted as critical components of successful service platforms. Informal service marketplaces often lack mechanisms for validating provider credibility, which reduces user confidence and increases uncertainty during transactions [13], [20]. Several studies emphasized the importance of verified provider profiles, ratings, reviews, and transparent communication systems in establishing trust between users and service providers. Platforms that integrate accountability mechanisms tend to improve user satisfaction and encourage continued platform usage because users are more confident when selecting providers with visible performance records. Nevertheless, some systems still fail to implement comprehensive trust mechanisms, particularly in localized or community-based environments where provider verification processes remain limited.

Another emerging practice in online service platforms is the integration of third-party identity verification services to strengthen provider authentication and improve platform security. Unlike traditional verification methods that rely solely on manual document checking by administrators, third-party verification services validate government-issued identification and authenticate user identities before allowing providers to offer services. Studies have shown that verification serves as a credible trust signal that reduces information asymmetry between service providers and customers, leading to better matching outcomes and increased user engagement [57]. Likewise, trust has been identified as one of the most important factors influencing users' continued use of online marketplace platforms, emphasizing the importance of verified identities in building customer confidence [58]. Furthermore, trust evaluation mechanisms help minimize malicious behavior and improve the selection of legitimate service providers, resulting in a safer and more reliable digital marketplace [59]. Therefore, integrating a third-party

verification agency into community-based service platforms can strengthen platform credibility, reduce fraudulent registrations, and improve the overall quality and security of service transactions.

Additional rising theme in the literature is sentiment analysis and opinion mining. Sentiment analysis refers to the process of analyzing textual data, such as reviews, comments, and feedback, to determine the emotional tone or attitude expressed by users. Researchers have shown that sentiment analysis techniques can be used to automatically classify feedback as positive, negative, or neutral, allowing organizations to better understand customer satisfaction and service quality. Compared to manual review evaluation, automated sentiment analysis enables faster processing of large volumes of user-generated content and provides more consistent assessment of customer opinions

Studies further indicate that sentiment analysis can enhance recommendation and decision-support systems by extracting meaningful insights from ratings and textual reviews. While numerical ratings provide quantitative measures of performance, sentiment analysis offers qualitative insights that help identify strengths, weaknesses, and recurring service issues. Many modern service platforms integrate sentiment analysis to improve transparency, monitor provider performance, and support data-driven management decisions. However, advanced sentiment analysis models often require large training datasets, extensive computational resources, and complex natural language processing techniques. For localized community-based systems, simpler sentiment analysis approaches may provide a practical alternative because they are easier to implement while still offering valuable insights into customer feedback and service quality.

Another major theme identified in the literature is the use of analytics and performance optimization. Chen et al. explained that analytics-driven systems can process user behavior, service demand patterns, and provider performance to improve operational efficiency and generate personalized recommendations [26].

Performance evaluation studies commonly measure response time, scalability, recommendation accuracy, and user satisfaction to determine system effectiveness [26]. Comparative analyses revealed that systems integrating adaptive analytics and intelligent algorithms generally achieve better scalability and efficiency than static or manually managed platforms [27]. These systems are capable of continuously improving service recommendations based on collected data, making them more responsive to changing user demands. However, analytics-driven platforms also require more sophisticated infrastructure and larger amounts of data, which may increase development complexity and operational costs.

In the Philippine context, the adoption of digital marketplace and service booking platforms has increased significantly due to the widespread use of smartphones and mobile internet technologies. Platforms such as Grab and Lalamove demonstrate how digital systems can improve service accessibility, facilitate transactions, and support local economic activity. Informal digital marketplaces, particularly Facebook community groups, are also widely used for hiring service providers and offering freelance services. Studies on Philippine digital platform adoption suggest that mobile-based systems improve convenience and expand economic opportunities for small-scale providers. However, most existing platforms focus mainly on transportation, food delivery, and e-commerce rather than localized household or community-based service bookings. Furthermore, many local systems still depend on unstructured communication methods and lack integrated features such as GPS-based matching, booking management, analytics, and verification systems.

The literature further highlights the importance of technology-driven service systems in supporting sustainable development. The United Nations emphasized that digital technologies contribute to Sustainable Development Goal (SDG) 8, which promotes decent work and economic growth, and SDG 11, which focuses on building sustainable and inclusive communities [9]. These objectives align with

the purpose of NeighborHelp in strengthening local economies and improving community engagement through digital innovation.

These gaps highlight the need for a comprehensive platform like NeighborHelp that integrates optimized service matching, real-time communication, booking management, GPS-based service discovery, third-party provider verification, trust mechanisms, sentiment analysis of user feedback, and analytics-driven monitoring into a scalable and user-centered community service platform.

Related Works

Several community service platforms and service discovery applications have been developed to improve local service accessibility and efficiency. These systems generally include real-time service browsing, in-app messaging, booking and scheduling, and rating/review mechanisms. Some advanced platforms integrate GPS-based location services and data analytics to optimize service matching and enhance operational efficiency [1], [9], [26], [28], [29].

Table 1. Comparison table of existing system vs. the proposed system

System	Service Browsing	In-app Messaging	Booking	GPS-Based Matching	Ratings and Review	Data Analytics
Thumbtack	✓	✓	✓	✓	✓	✗
TaskRabbit	✓	✓	✓	✓	✓	✗
Angi	✓	✓	✓	✗	✓	✗
Bark	✓	✓	✓	✗	✓	✗
MyKuya	✓	✓	✓	✓	✓	✗

Neighborhelp (Proposed System)	✓	✓	✓	✓	✓	✓
---	---	---	---	---	---	---

Table X compares NeighborHelp with existing commercial service marketplace platforms. Similar platforms such as Thumbtack, TaskRabbit, Angi, Bark, and MyKuya provide essential features including service browsing, communication between customers and providers, booking management, and customer reviews. However, these platforms primarily focus on facilitating service transactions and generally do not provide an integrated analytics dashboard for administrators to monitor booking trends, provider performance, verification statistics, and customer feedback within a localized community setting. NeighborHelp addresses this gap by combining GIS-based provider matching, provider verification, sentiment analysis, and an analytics dashboard into a single platform designed for community-based service management [26].

Definition of Terms

Booking and Scheduling – A system feature that allows users to request, reserve, and schedule appointments with service providers at preferred dates and times.

Community-Based Platform – An online platform that connects people within a specific community to access local services, share resources, and interact with trusted service providers.

Customer / User – A community member who uses the NeighborHelp application to browse, book, and evaluate services from registered providers.

Data Analytics – The collection, processing, and analysis of platform-generated data, such as service searches, booking frequency, ratings, and user behavior, to

generate insights that improve service matching, provider performance, and overall system efficiency.

GPS-Based Location Services – Technology that utilizes Global Positioning System data to identify the physical location of users and providers, enabling accurate and efficient service recommendations.

In-App Messaging – A communication feature within the NeighborHelp platform that allows direct messaging between customers and service providers, enabling coordination and clarification of service requirements.

NeighborHelp – A community-centered mobile and web application that connects residents with local service providers, enabling service discovery, booking, communication, and feedback in a centralized platform.

Real-Time Service Matching – The process of dynamically identifying and presenting nearby service providers to a customer based on their location and service requirements using GPS or geolocation features.

Rating and Review System – A feedback mechanism that enables customers to evaluate the quality of service provided, share experiences, and help other users make informed decisions.

Role-Based Access – A security feature in the NeighborHelp platform that grants users specific permissions and access based on their role, such as customer, service provider, or administrator.

Service Provider – An individual or business offering a service to the community, such as tutors, cleaners, electricians, or plumbers, who can register on the NeighborHelp platform to provide and manage their services.

CHAPTER 2

METHODOLOGY

This chapter presents the methodology used in the development of NeighborHelp: A Community-Based Service Finder and Booking Platform. It adopts a Hybrid Project Management Methodology under the PADIM framework (Planning, Analysis, Design, Implementation, and Maintenance). The methodology combines the structured and sequential approach of the Waterfall model with the iterative and flexible practices of Agile development. The Waterfall model emphasizes systematic planning, documentation, and organized development [31], while Agile promotes iterative development, continuous testing, and stakeholder feedback [32]. Combining these approaches provides both stability and adaptability throughout the project lifecycle [33].

During the Planning Phase, the project objectives, scope, timeline, resources, stakeholders, and potential risks are identified. To reduce these risks, the proponents apply proper scheduling, task monitoring, regular adviser consultations, backup planning, and continuous review of technical requirements. The Analysis Phase focuses on gathering and validating the functional and nonfunctional requirements of customers, service providers, and administrators through consultations and feedback activities.

In the Design Phase, the system requirements are translated into technical designs, including the system architecture, database structure, interface layouts, and workflow diagrams. Proposed changes are evaluated and documented before implementation.

The Implementation Phase follows an Agile approach consisting of four sprints. Sprint 1 develops user registration and authentication features. Sprint 2 implements service listing, search functions, and GPS-based provider matching.

Sprint 3 focuses on booking management and in-app messaging. Sprint 4 delivers ratings and reviews, notifications, analytics, reporting, and administrative monitoring features. Sprint reviews and retrospectives are conducted after each cycle to gather feedback and support continuous improvement, consistent with Agile development practices [32].

Finally, the Maintenance Phase ensures the system remains secure, reliable, and functional through bug fixes, performance monitoring, security updates, user feedback evaluation, and preparation for future enhancements. User acceptance testing is also conducted to verify that the system satisfies stakeholder requirements.

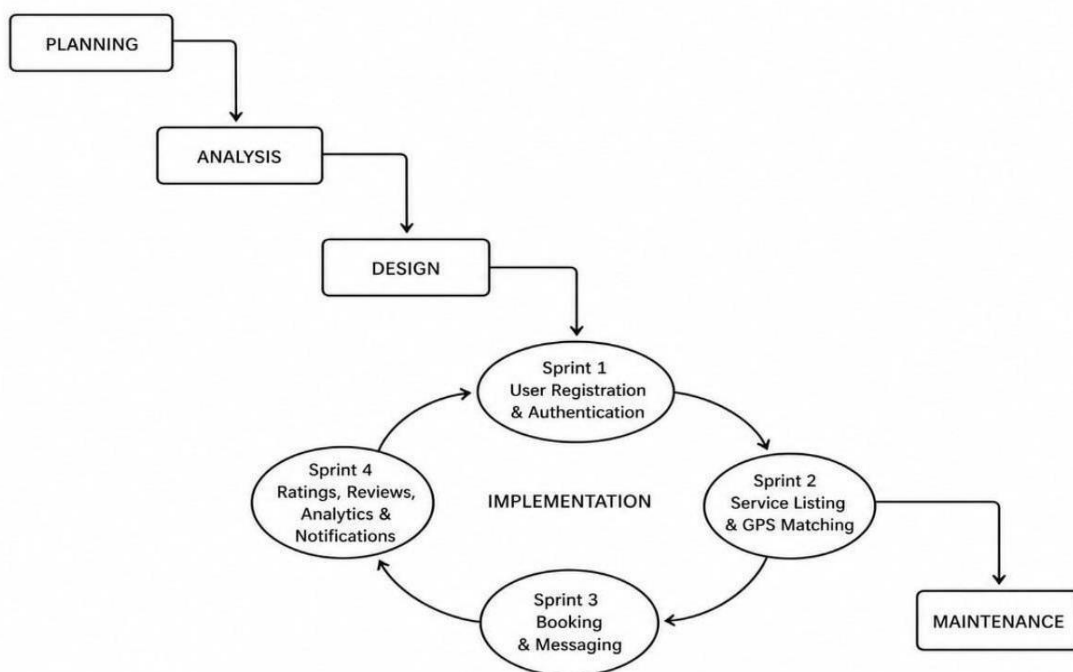


Figure 1: Integrated Hybrid Project Management Approach

SYSTEM PLANNING

Project Team Organization

The project team is organized to ensure clear responsibility, proper coordination, and efficient completion of the NeighborHelp system. The Capstone Adviser provides academic and technical guidance, checks the quality of project outputs, and ensures that the study follows the required standards for capstone development. The Project Manager oversees the project schedule, monitors task progress, coordinates team meetings, and ensures that deliverables are completed on time.

The System Developer is responsible for coding, debugging, database integration, Firebase configuration, and implementation of the system modules. The System Analyst and Documentation Specialist handles requirements analysis, system documentation, diagram preparation, and organization of project records. This structure allows the team to balance both technical development and documentation tasks while maintaining accountability throughout the project lifecycle.

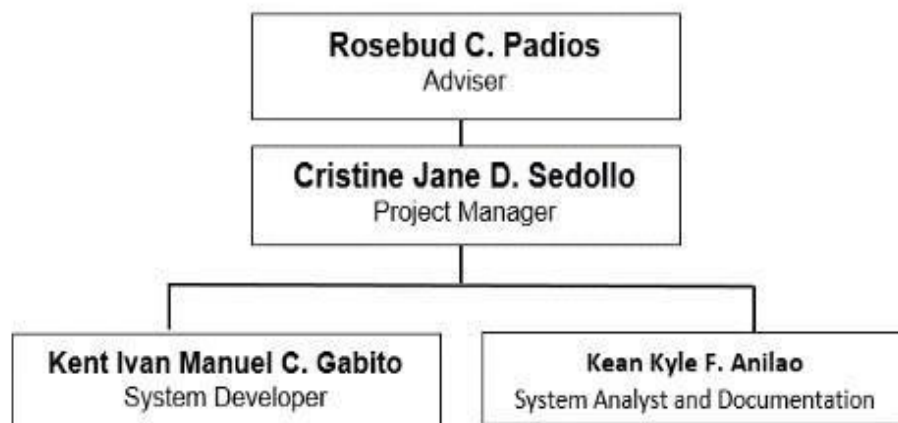


Figure 2. Project Management Team Organization

Work Breakdown Structure

The Work Breakdown Structure (WBS), as presented in Figure 3, is structured under five PADIM phases, each broken down into specific work packages that collectively cover the full development lifecycle of NeighborHelp.

The Planning phase contains work packages for Project Planning and Scheduling, Team Coordination and Meetings, Progress Monitoring and Reporting, and Documentation and Submission, focusing on managing the project as a whole rather than building the system itself.

The Analysis phase includes Stakeholder Requirement Gathering, System Requirements Specification, Related Systems Review, and Use Case and DFD Diagrams. These activities establish a thorough understanding of the problem before any design or coding begins.

The Design phase is the most detailed, containing work packages for Database Schema Design, System Architecture Design, UI and UX Wireframes, GPS Matching Module Design, and Messaging Module Design, reflecting the technical complexity of the platform's core features.

The Implementation phase is organized into four sprint-aligned work packages: User Registration and Authentication, Service Listing and GPS Matching, Booking and Messaging, and Analytics and Notifications. Each sprint builds on the previous one in a logical sequence.

Finally, the Maintenance phase covers Deployment and Configuration, User Orientation and Training, System Monitoring and Support, and Continuous Improvement, ensuring that NeighborHelp is not merely built and submitted but is delivered and sustained for the community it serves.

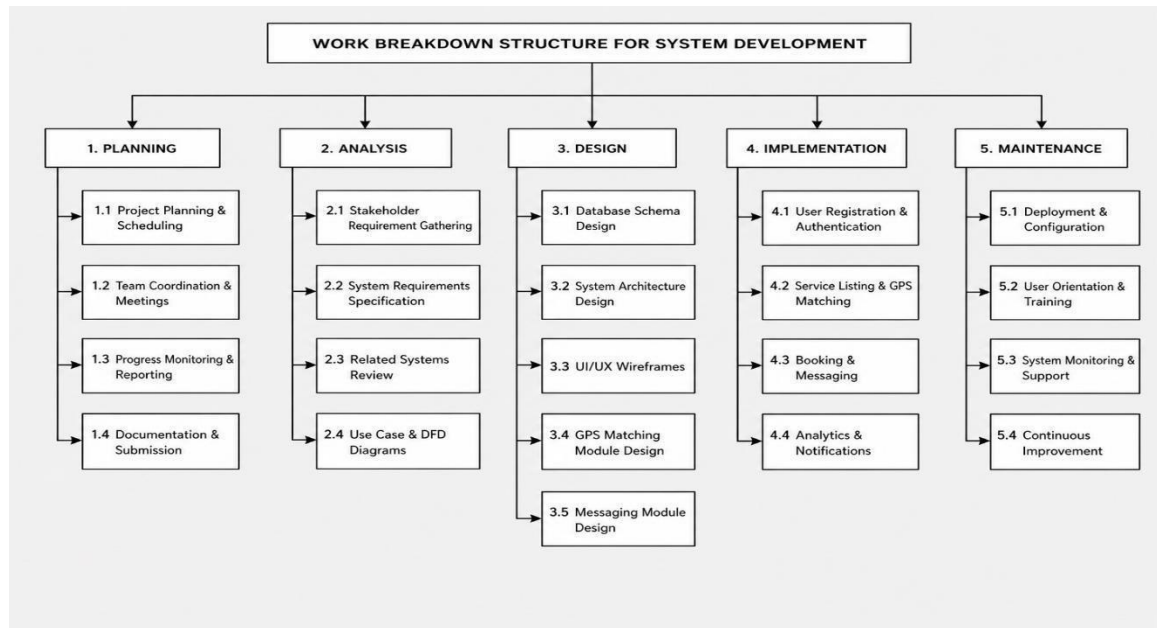


Figure 3: Work Breakdown Structure

Gantt Chart

The Gantt chart, as presented in Figure 4, illustrates the timeline and schedule of activities for the development of the NeighborHelp system, aligned with the PADIM framework. It shows the sequence and duration of each phase, from Planning and Analysis to Design, Implementation, and Maintenance, ensuring that all activities such as requirement finalization, system design, coding, testing, and report generation are completed within the specified timeframe. The chart helps the proponents monitor progress, manage time effectively, and ensure that each phase is completed on schedule [34], [35].



Figure 4. Gantt Chart of the system

SYSTEM ANALYSIS

This section discusses the process by which the NeighborHelp: A Community-Based Service Finder and Booking Platform was thoroughly analyzed to address the existing challenges in local service access and community coordination. The study identified key problems including the absence of a structured digital channel for finding nearby service providers, lack of real-time communication between customers and providers, no transparent rating and review mechanism, and limited visibility for small-scale local workers. The analysis involved assessing stakeholder needs, system requirements, and existing workflows to ensure that the proposed system effectively responds to these challenges.

Through this analysis, essential system features were determined, including a centralized service discovery and booking platform, GPS-based provider matching for location-aware search results, role-based access control for secure and organized data management, real-time in-app messaging for direct

communication, and an analytics dashboard for data-driven monitoring. These components were carefully designed to improve accessibility, transparency, and efficiency of local service transactions, ensuring that the system is both practical and sustainable for use by community members and local service providers.

System Architecture

Figure 5 presents the system architecture of NeighborHelp, a centralized and layered architecture that integrates service discovery, GPS-based provider matching, booking management, messaging, notifications, ratings and reviews, analytics, and administrative monitoring. The system supports four user roles: Customer, Service Provider, Verification Agency and System Administrator, with access controlled through Firebase Authentication and role-based authorization.

The architecture consists of three main components: Technology Stack, Core Modules, and Data Layer. The technology stack includes Flutter, Dart, Firebase services, Google Maps Platform, and supporting libraries. The core modules handle authentication, user and provider management, service listings, GPS-based matching, booking and scheduling, messaging, ratings and reviews, notifications, analytics, and activity logging.

The Data Layer uses Cloud Firestore to store users, providers, services, bookings, messages, reviews, notifications, and activity logs, while Firebase Storage manages uploaded files such as profile photos, service images, verification documents, and message attachments. Real-time synchronization ensures that updates are immediately reflected across connected devices.

The architecture also includes supporting infrastructure for backup and recovery, offline data synchronization, and scalability. Automated backups, local caching, and synchronization mechanisms improve reliability, while Firebase's cloud services support future expansion and increased user demand.

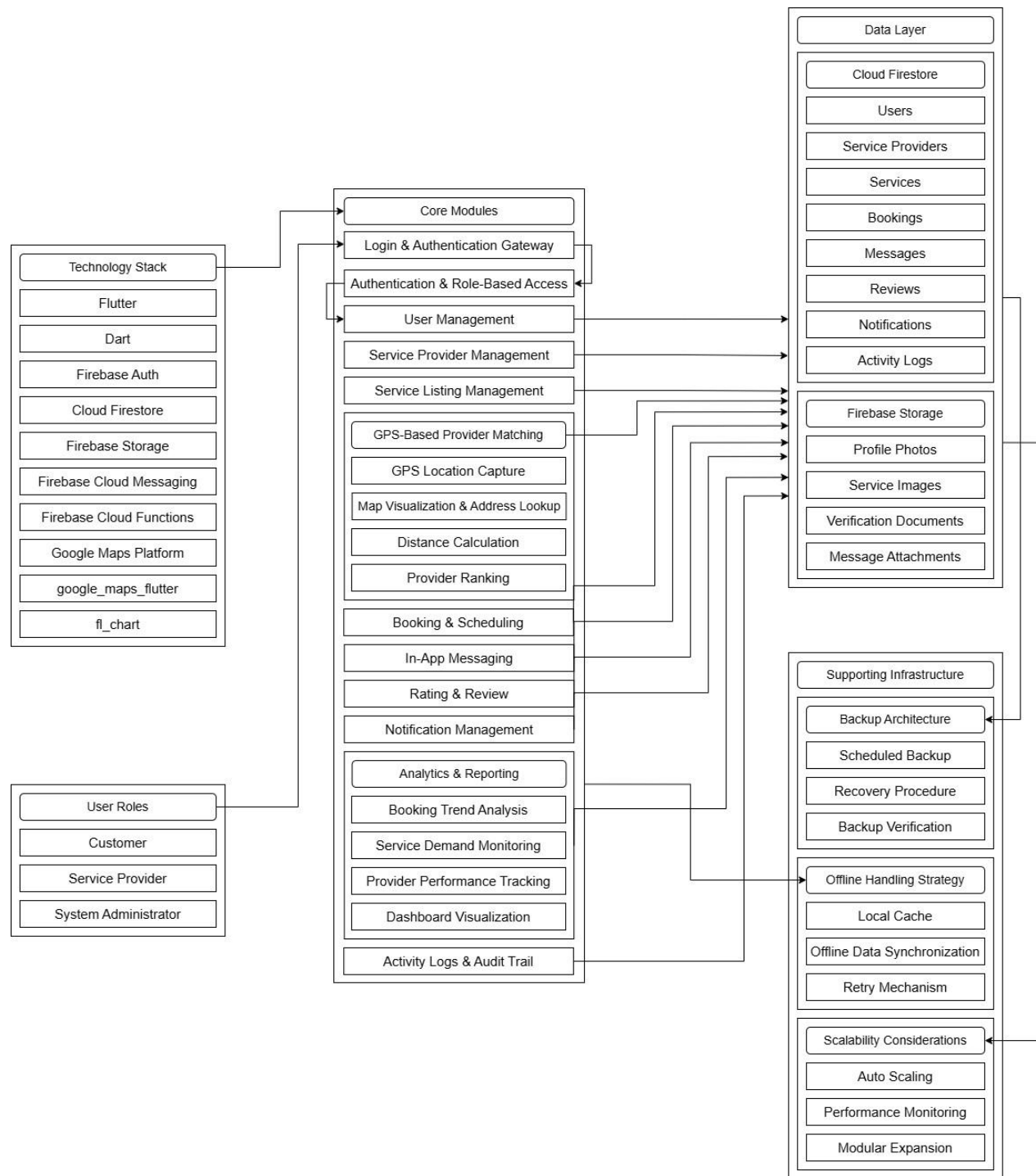


Figure 5: System Architecture of NeighborHelp

Functional Requirements

The NeighborHelp system is designed to support comprehensive service discovery, GPS-based provider matching, real-time communication, structured booking management, and notifications, ratings and reviews, and data analytics reporting. The system ensures accurate data management, efficient service transactions, and transparent communication between customers, service providers, and administrators.

Modules and Their Functions

- 1. Service Provider Location Mapping.** The system shall enable service providers to register their service locations and display them on an interactive GIS map, allowing households to locate nearby providers based on geographic proximity and service category.
- 2. Location-Based Search.** The system shall allow households to search and filter service providers based on location, service type, and service coverage area.
- 3. Service Request Management.** The system shall allow households to submit service requests containing their required service details.
- 4. Intelligent Provider Matching.** The system shall automatically recommend suitable service providers by evaluating service requirements, geographic proximity, provider ratings, availability, and completion rate. Only verified service providers shall be included in the recommendation process.
- 5. Booking and Request Handling.** The system shall facilitate booking transactions by allowing households to hire providers and enabling providers to accept or decline service requests.
- 6. Provider Verification.** The system shall allow service providers to submit government-issued identification and other required documents for identity verification through an integrated third-party verification agency.
- 7. Verification Management.** The system shall receive verification results from the integrated third-party verification agency, automatically update the

provider's verification status, and display verification badges on verified provider profiles. Administrators shall be able to monitor verification records, view the verification results provided by the third-party verification agency, and manually review exceptional cases, such as disputed or incomplete verification requests, when necessary.

- 8. Provider Service Profile Review.** The system shall require verified service providers to complete their service profiles before offering services on the platform. The administrator shall review each service profile to ensure that the selected service category, service description, pricing information, service area, and supporting documents comply with the platform's policies and standards. Only approved service profiles shall be made visible to customers for booking.
- 9. Verification Status Notification.** The system shall notify service providers of their verification status (e.g., Pending, Verified, or Rejected) after the verification process is completed.
- 10. Feedback Collection.** The system shall allow households to submit ratings and reviews after service completion.
- 11. Sentiment Analysis.** The system shall analyze user feedback and classify reviews as positive, neutral, or negative to generate provider reputation indicators.
- 12. Service Analytics Dashboard.** The system shall provide an analytics dashboard that visualizes booking activities, provider performance, verification statistics, service demand, and sentiment analysis results through charts, graphs, and GIS-based reports.
- 13. User Account Management.** The system shall allow households, service providers, and administrators to register, authenticate, and manage their accounts.
- 14. Notification Management.** The system shall send notifications related to service requests, bookings, verification updates, and completed transactions.

15. Report Generation. The system shall generate and export reports related to service transactions, provider performance, verification status, and user feedback analytics.

Non-Functional Requirements

These requirements define the quality attributes of the NeighborHelp system in accordance with the ISO/IEC 25010 standard. They ensure that the system achieves high levels of functional suitability, usability, reliability, security, maintainability, and portability while meeting the needs of customers, service providers, and administrators.

1. Functional Suitability

1.1 The system shall provide complete and accurate functionalities for service discovery, GPS-based provider matching, booking management, in-app messaging, notifications, ratings and reviews, and analytics reporting.

1.2 The system shall ensure the correctness of processed data, including user profiles, service listings, booking records, location data, message records, notification records, and review scores.

1.3 The system shall support the required tasks of customers, service providers, and administrators without requiring separate external tools for core platform operations.

2. Usability

2.1 Users shall easily recognize system functions through clear labels, intuitive navigation, and consistent interface design on both the Flutter mobile application and Flutter Web administrative dashboard.

2.2 New users shall be able to complete basic tasks such as registering, searching for a service, viewing provider profiles, and submitting a booking request within a reasonable amount of time after first use.

2.3 The system shall provide role-specific screens and dashboards tailored to the workflows of customers, service providers, and administrators.

2.4 The system shall validate user inputs and display clear, user-friendly error messages to prevent incorrect or incomplete data submission.

2.5 The interface shall be responsive, organized, and accessible across supported mobile devices and web browsers.

3. Reliability

3.1 The system shall operate with minimal errors during normal use.

3.2 The system shall maintain stable access to core services such as authentication, service browsing, booking management, messaging, and notifications during operational use.

3.3 The system shall handle failures of external services such as Google Maps Platform and Firebase Cloud Messaging gracefully, without crashing or losing important user data.

3.4 The system shall support data backup and recovery through Firebase and Cloud Firestore configuration to reduce the risk of data loss.

4. Security

4.1 Sensitive user data, booking records, message records, and location information shall be protected from unauthorized access through Firebase Authentication, Cloud Firestore security rules, Firebase Storage security rules, and role-based access control.

4.2 All data transmitted between the Flutter mobile applications, Flutter Web dashboard, Firebase services, and Google Maps Platform shall use secure communication protocols such as HTTPS and Transport Layer Security (TLS).

4.3 User actions such as logins, booking updates, profile changes, and administrative actions shall be logged where applicable to support accountability and auditability.

4.4 The system shall follow the principles of the Philippine Data Privacy Act of 2012 (RA 10173) and adhere to relevant ISO/IEC 27001 security practices for protecting personal and location-based information.

5. Maintainability

5.1 The system shall be designed using modular components such as the authentication module, GPS matching module, booking module, messaging module, notification module, analytics module, and review module to allow independent updates and easier maintenance.

5.2 System components shall be reusable to support future enhancements such as adding new service categories, expanding to additional localities, improving provider verification, or integrating online payment features.

5.3 System logs, error messages, and development documentation shall support efficient debugging, issue tracking, and future system improvements.

6. Portability

6.1 The system shall operate on Android mobile devices through the Flutter mobile application and on major web browsers through the Flutter Web administrative dashboard.

6.2 The system may be extended to iOS devices depending on deployment requirements, available resources, and future implementation plans.

6.3 The system shall be deployable using cloud-based Firebase services to support future scaling, maintenance, and migration to additional communities or service areas.

Use Case Diagram

The Use Case Diagram of the NeighborHelp system illustrates the interactions between the system's four primary actors: Customer, Service Provider, Verification Agency and System Administrator,. Each actor performs specific functions based on their respective roles and responsibilities within the platform. Each actor is assigned specific responsibilities that correspond to the system's core functionalities.

The Customer interacts with the system through the Flutter mobile application. The customer can register an account, log in, manage their profile, search for available service providers, book services, track booking status, send and receive messages, receive notifications, and submit ratings and reviews after a completed service. These functionalities enable customers to conveniently locate trusted service providers, communicate with them, and evaluate the quality of services received.

The Service Provider also accesses the system using the Flutter mobile application. Service providers can register and log in, manage their profiles, create and update service listings, manage their availability, accept or reject booking requests, send and receive messages, receive notifications, and submit verification documents for identity and credential verification. These functions allow service providers to manage their offered services while ensuring that they meet the platform's verification requirements.

The Verification Agency is responsible for validating the legitimacy of service providers. Through the verification interface, the agency reviews the submitted verification documents, verifies the service provider's identity and credentials, approves or rejects verification requests, and sends verification results. This process ensures that only legitimate and qualified service providers are recognized as verified members of the platform, thereby improving customer trust and platform reliability.

The System Administrator accesses the system through the Flutter Web administrative dashboard. The administrator manages user accounts, monitors the verification status of service providers, oversees booking activities, generates reports, and manages overall system records. Unlike the Verification Agency, the administrator does not perform the verification itself but supervises the verification workflow and ensures the efficient operation, security, and integrity of the NeighborHelp platform.

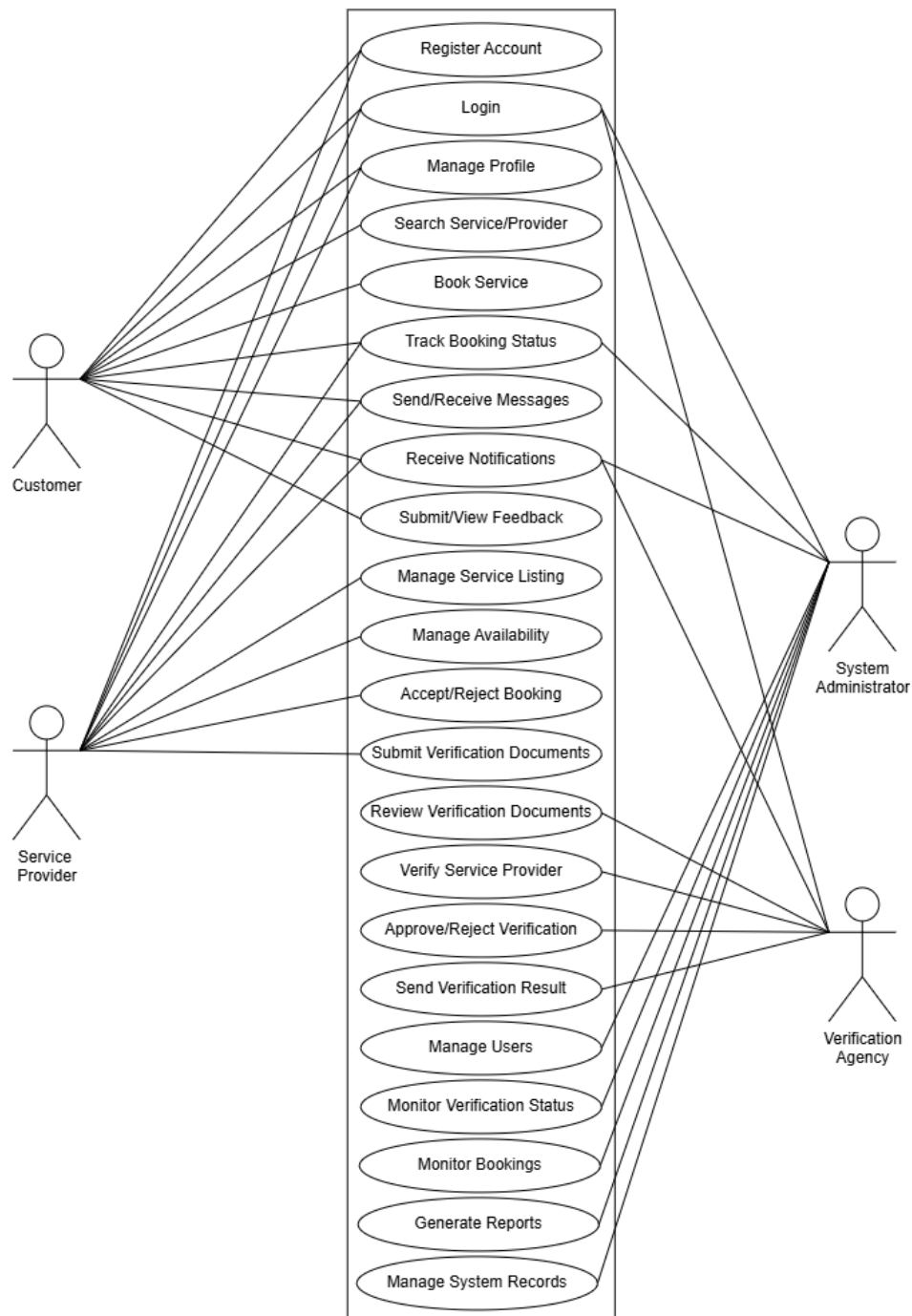


Figure 6. Use Case Diagram of the System

Context Flow Diagram

The Context Flow Diagram, shown in Figure 7, presents a high-level overview of the NeighborHelp: A Community-Based Service Finder and Booking Platform by illustrating how the system exchanges information with its four external entities: the Customer, Service Provider, Verification Agency, and System Administrator. The diagram highlights the major data inputs and outputs that enable the platform to support service booking, communication, provider verification, and system administration.

The Customer provides inputs such as personal information, service requests, booking request details, message details, feedback details, and report details. In return, the system provides verified account information, provider information, booking information, message information, notification information, feedback information, and report information. These data exchanges allow customers to search for verified service providers, book services, communicate with providers, monitor booking progress, and submit reviews or reports after a completed service.

The Service Provider provides inputs such as profile information, service listing details, availability schedules, service area information, booking responses, and in-app messages. In return, the Service Provider receives outputs such as booking request notifications, customer messages, booking status updates, customer reviews, and provider performance summaries.

The Verification Agency is responsible for validating the legitimacy of service providers. The system sends provider information and verification documents to the Verification Agency for evaluation. After reviewing the submitted documents, the agency returns verification results, verification status, and verification remarks to the system. This verification process helps ensure that only legitimate and qualified service providers are recognized as verified members of the NeighborHelp platform.

The System Administrator manages the overall operation of the platform by providing provider verification details, system verification decisions, notification details, and report generation requests. In return, the administrator receives account information, analytics reports, report information, verification information, and notification information. These data exchanges enable the administrator to oversee user accounts, monitor verification activities, generate reports, and maintain the overall integrity and performance of the system.

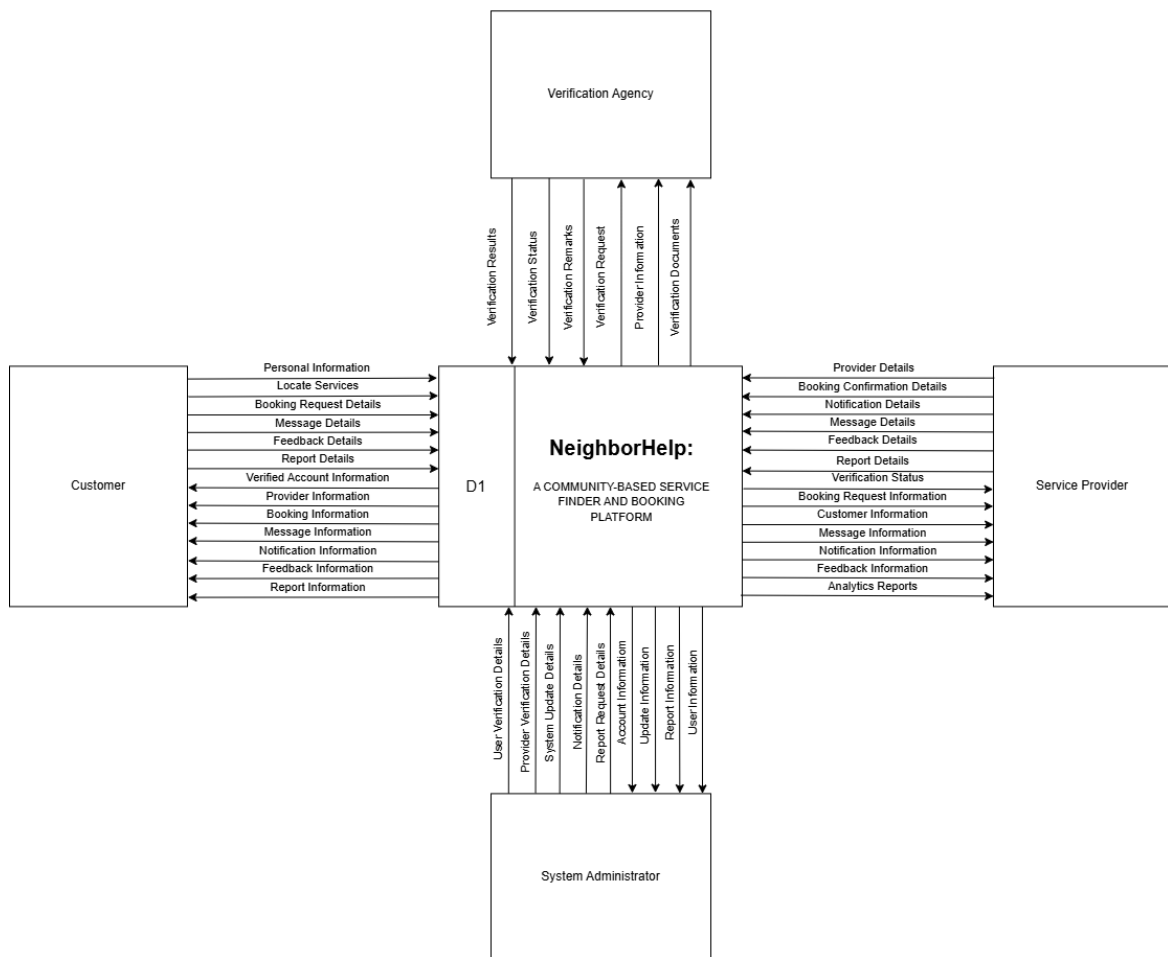


Figure 7. Context Flow Diagram of the System

Data Flow Diagram

The Level 1 Data Flow Diagram (DFD), shown in Figure 8, provides a more detailed representation of the NeighborHelp system by expanding the major processes identified in the Context Flow Diagram. It illustrates how data moves between the Customer, Service Provider, System Administrator, Verification Agency, and the NeighborHelp Database (Cloud Firestore) to support the platform's core functionalities..

The system consists of six primary processes: User and Role Management (1.0), Verification Management (2.0), Service and Provider Management (3.0), Booking and Scheduling (4.0), Communication and Notifications (5.0), and Feedback and Reports (6.0).

The User and Role Management (1.0) process manages user registration, login, account management, and role assignment for customers, service providers, and system administrators. Customers and service providers submit their personal information, provider details, and account details, while administrators manage account information and user information. The process stores verified account information and user records in the NeighborHelp Database, allowing authorized users to securely access and manage their accounts according to their assigned roles.

The Verification Management (2.0) process is responsible for validating the legitimacy of service providers. The system forwards user verification details and provider verification details to the Verification Agency, which reviews the submitted information and returns verified account information and user verification results. These verification records are stored in the NeighborHelp Database and are used to identify verified service providers within the platform, helping improve customer trust and service reliability.

The Service and Provider Management (3.0) process manages provider profiles, service listings, and service information. Service providers submit their

provider details, while customers retrieve provider information and available services when searching for service providers. The System Administrator also manages user information and system update details. All provider and service information is stored and updated in the NeighborHelp Database for easy access and management.

The Booking and Scheduling (4.0) process manages booking transactions between customers and service providers. Customers submit booking requests containing service details and preferred schedules. Service providers receive booking requests and return booking confirmation details after accepting or rejecting the requests. The process stores booking request information, booking confirmations, and booking customer information in the NeighborHelp Database to ensure accurate tracking and synchronization of booking records.

The Communication and Notifications (5.0) process supports communication between customers and service providers by managing message exchanges and notification updates. Both customers and service providers send message details and notification details to the system, while receiving message information and notification information in return. These communication records are stored in the NeighborHelp Database to maintain real-time coordination throughout the booking process.

The Feedback and Reports (6.0) process collects feedback, ratings, reviews, and reports submitted by customers and service providers after a completed service. The System Administrator may also submit report requests for monitoring and evaluation purposes. Feedback information and report information are stored in the NeighborHelp Database, enabling administrators to monitor service quality, evaluate provider performance, address reported concerns, and generate reports that support continuous improvement of the NeighborHelp platform.

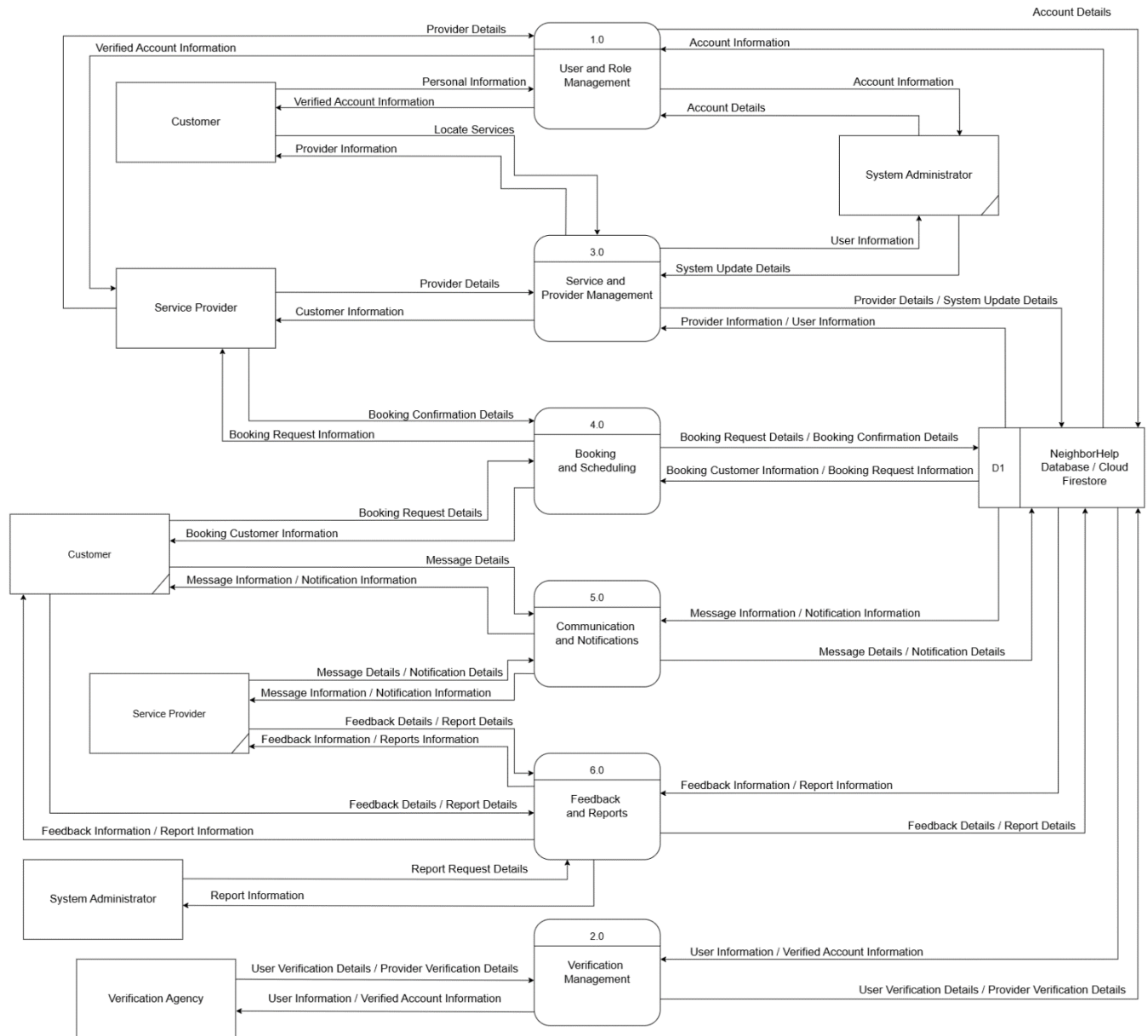


Figure 8. Level 1 Data Flow Diagram of the System

SYSTEM DESIGN

Entity Relationship Diagram

The Entity Relationship Diagram, illustrated in Figure 9, presents the logical structure of the NeighborHelp database, showing how major system data collections are organized and related to one another. Since NeighborHelp uses Cloud Firestore as its main cloud database, the system stores records in collections and documents rather than traditional relational tables. However, the ERD still serves as a useful guide for understanding the relationships among the major data components of the platform.

The major data collections of NeighborHelp include Users, Service Providers, Services, Bookings, Messages, Reviews, Notifications and Verifications. These collections support the core functions of the platform, including account management, service discovery, booking coordination, real-time messaging, ratings and reviews, and system notifications.

The Users collection stores identity and profile information for all registered accounts, including customers, service providers, and administrators. Each user document contains details such as full name, email address, contact number, role, account status, profile photo, and timestamps. The user role determines the level of access granted to the account.

The Service Providers collection stores provider-specific information linked to user accounts with the Service Provider role. It contains provider details such as biography, service area, location coordinates, verification status, average rating, and profile image. A service provider may create multiple service listings, forming a one-to-many relationship between service providers and services.

The Services collection stores the services offered by registered providers. Each service document contains the service title, category, description, estimated price, duration, availability, status, and related provider ID. These records are used by the service discovery and GPS-based matching modules when customers search for nearby providers.

The Bookings collection stores service transaction records between customers and providers. Each booking document contains the customer ID, provider ID, service ID, selected schedule, service location, booking status, total fee, and additional instructions. A booking record connects the customer, provider, and selected service into one transaction.

The Messages collection stores real-time conversation records between customers and service providers. Each message is linked to a specific booking to keep communication organized and traceable. Message records may include the sender ID, message content, media attachment URL, read status, and timestamp.

The Reviews collection stores feedback submitted by customers after completed bookings. Each review is connected to a booking, customer, provider, and service. It contains the star rating, written comment, and date submitted. Review records support transparency and help improve provider credibility within the platform.

The Notifications collection stores system-generated alerts delivered to users. These may include booking requests, booking confirmations, cancellations, new message alerts, appointment reminders, and review requests. Each notification is linked to a user recipient and includes the notification type, content, read status, and timestamp.

The Verification collection stores identity verification records of service providers through integration with the Didit identity verification system. Each verification record is linked to a provider account and contains details such as the type of identification submitted, uploaded ID documents, selfie verification image, Didit session ID, verification result, and verification status. It also records administrative review information such as the reviewer ID and review timestamp. This collection ensures that service providers are properly validated before being allowed to offer services on the platform.

Relationships between these collections are maintained through document IDs and reference fields such as `userId`, `providerId`, `serviceId`, `bookingId`, and

customerId. Through this structure, NeighborHelp can efficiently retrieve user records, service listings, booking transactions, message threads, reviews, and notifications while supporting real-time updates through Cloud Firestore.

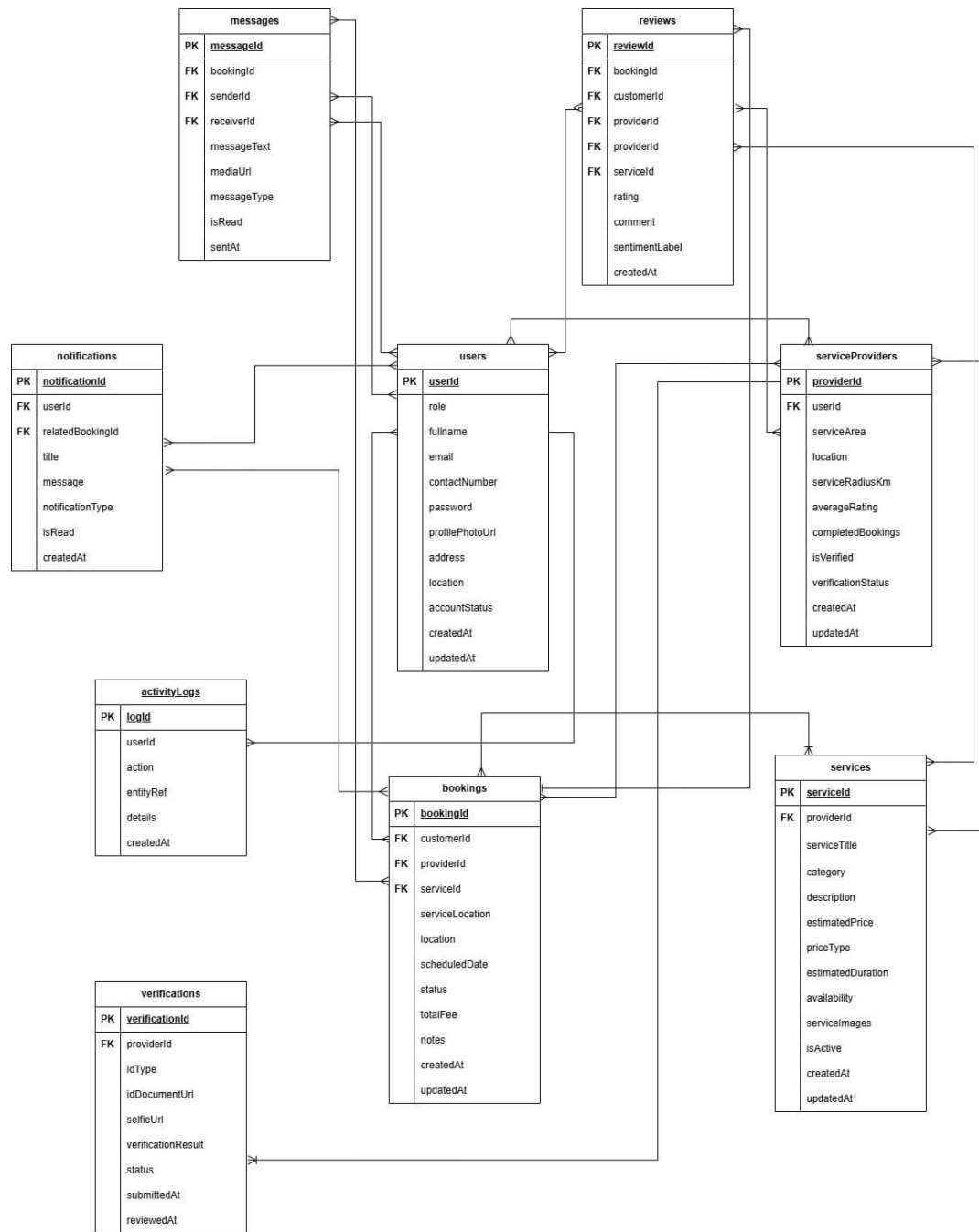


Figure 9: Entity Relationship Diagram of the System

Data Dictionary

The data dictionary provides a detailed description of every major collection defined in the NeighborHelp Cloud Firestore database, including their fields, data types, constraints, and purpose. It serves as a reference document that complements the Entity Relationship Diagram by clarifying how each piece of information is stored, validated, and related to other records. The data dictionary ensures that developers, administrators, and stakeholders share a consistent understanding of the database structure, which is important for maintaining data integrity, supporting future development, and enabling accurate reporting and analytics within the system.

Users

Table 1 stores all registered users of the system, including customers, service providers, and administrators. It contains essential personal information, account details, role classification, and activity tracking data. Each user is uniquely identified through a system-generated `userId` provided by Firebase Authentication. The table supports communication, authentication, role-based access control, and user profile management. Optional fields such as contact number, address, and location enhance service personalization, while timestamps are used to monitor account creation, updates, and user activity. The `accountStatus` field ensures proper management of user accessibility within the platform.

Table 1: Data Dictionary for Users Entity

Field Name	Data Type	Field Length	Constraint	Description
userId	String	20	Document ID, Unique	Unique identifier of the user account generated by Firebase Authentication.
fullName	String	100	Required	Complete name of the customer, service provider, or administrator.

email	String	100	Required, Unique	Email address used for login and account communication.
contactNumber	String	11	Optional	Mobile number of the user for communication and account reference.
role	String	50	Required	Assigned role of the user: Customer, Service Provider, or Administrator.
profilePhotoUrl	String	100	Optional	URL of the uploaded profile photo stored in Firebase Storage.
address	String	100	Optional	Saved address or general location of the user.
latitude	Decimal	10,8	Optional	Stores the latitude coordinate.
longitude	Decimal	11,8	Optional	Stores the longitude coordinate.
accountStatus	String	50	Required	Status of the account, such as Active, Pending, Suspended, or Deactivated.
createdAt	Timestamp	-	Required	Date and time when the account was created.
updatedAt	Timestamp	-	Optional	Date and time when the account was last updated.

Service Providers

This table contains detailed information about users who are registered as service providers within the system. Each record is uniquely identified by providerId and is linked to the corresponding user account through userId, ensuring integration with the Users table. The table stores professional details such as the provider's

biography, service area, and geographic location, which are essential for matching providers with customers based on proximity and service coverage.

Operational metrics such as `averageRating` and `completedBookings` are maintained to evaluate provider performance and reliability. The `serviceRadiusKm` field defines the maximum distance within which the provider is willing to offer services, supporting location-based filtering. Verification-related fields, including `isVerified` and `verificationStatus`, are used to manage trust and platform credibility by indicating whether a provider has been reviewed and approved by an administrator. Timestamps such as `createdAt` and `updatedAt` are included to track the lifecycle and updates of each provider profile.

Table 2: Data Dictionary for Service Providers Entity

Field Name	Data Type	Field Length	Constraint	Description
<code>providerId</code>	String	20	Document ID, Unique	Unique identifier of the service provider record.
<code>userId</code>	String	20	Required, Reference	References the <code>userId</code> of the provider's account in the Users collection.
<code>bio</code>	String	100	Optional	Brief description of the provider and their offered services.
<code>serviceArea</code>	String	50	Required	Text description of the provider's covered service area.
<code>latitude</code>	Decimal	10,8	Optional	Stores the latitude coordinate.
<code>longitude</code>	Decimal	11,8	Optional	Stores the longitude coordinate.

serviceRadiusKm	Number	-	Optional	Maximum distance in kilometers where the provider is willing to offer services.
averageRating	Number	-	Default 0.00	Aggregated provider rating based on customer reviews.
completedBookings	Number	-	Default 0	Number of successfully completed bookings by the provider.
isVerified	Boolean	-	Default false	Indicates whether the provider has been reviewed or verified by the administrator.
verificationStatus	String	50	Required	Status of verification, such as Pending, Approved, or Rejected.
createdAt	Timestamp	-	Required	Date and time when the provider profile was created.
updatedAt	Timestamp	-	Optional	Date and time when the provider profile was last updated.

Services

Table 3 provides an overview of the Services table utilized in the system. This table represents the data component responsible for storing all service listings offered by registered service providers. Each record corresponds to a specific service and includes essential information such as the service title, category, description, pricing details, and estimated duration.

The table also supports operational functionality by linking each service to a provider through the providerId, enabling proper association and management of service ownership. Additional attributes such as availability schedules and service images enhance user experience by providing more detailed and visual information

about the offered services. Furthermore, fields like `isActive` allow control over the visibility of services, while timestamp attributes track the creation and modification of each listing.

Table 3: Data Dictionary for Services Entity

Field Name	Data Type	Field Length	Constraint	Description
<code>serviceId</code>	String	20	Document ID, Unique	Unique identifier of each service listing.
<code>providerId</code>	String	20	Required, Reference	References the <code>providerId</code> of the service provider who owns the listing.
<code>serviceTitle</code>	String	50	Required	Name or title of the service offered.
<code>category</code>	String	50	Required	Service category such as Cleaning, Plumbing, Electrical Repair, Tutoring, Caregiving, or Errand Assistance.
<code>description</code>	String	50	Required	Detailed description of the service being offered.
<code>estimatedPrice</code>	Number	-	Required	Estimated price or starting cost of the service.
<code>priceType</code>	String	50	Optional	Indicates whether the price is fixed, hourly, per visit, or negotiable.
<code>estimatedDuration</code>	String	50	Optional	Estimated time needed to complete the service.
<code>availability</code>	Map	-	Optional	Provider's available days and times for this service.
<code>serviceImages</code>	Array	-	Optional	List of image URLs related to the service listing.

isActive	Boolean	-	Default true	Indicates whether the service listing is visible and available for booking.
createdAt	Timestamp	-	Required	Date and time when the service listing was created.
updatedAt	Timestamp	-	Optional	Date and time when the service listing was last updated.

Bookings

Table 4 provides an overview of the Bookings table utilized in the system. This table represents the data component responsible for managing all service transactions between customers and service providers. Each record corresponds to a specific booking and includes essential information such as the customer, provider, and selected service through their respective reference IDs.

The table stores important transaction details, including the service location, scheduled date and time, and the current booking status, which allows the system to track the progress of each request from initiation to completion. Additional fields such as `totalFee` and `notes` provide flexibility for pricing agreements and special instructions from customers. Location data using GPS coordinates enhances mapping and navigation features, while timestamp fields track when the booking was created and last updated.

Table 4: Data Dictionary for Bookings Entity

Field Name	Data Type	Field Length	Constraint	Description
bookingId	String	20	Document ID, Unique	Unique identifier of each booking transaction.
customerId	String	20	Required, Reference	References the userId of the customer who requested the service.

providerId	String	20	Required, Reference	References the providerId of the selected service provider.
serviceId	String	20	Required, Reference	References the serviceId of the booked service.
serviceLocation	String	50	Required	Address or location where the service will be performed.
latitude	Decimal	10,8	Optional	Stores the latitude coordinate.
longitude	Decimal	11,8	Optional	Stores the longitude coordinate.
scheduledDate	Timestamp	-	Required	Preferred date and time requested by the customer.
status	String	50	Required	Booking status such as Pending, Accepted, Declined, Completed, Cancelled, or Rescheduled.
totalFee	Number	-	Optional	Agreed or estimated total fee for the booking transaction.
notes	String	50	Optional	Additional instructions or remarks from the customer.
createdAt	Timestamp	-	Required	Date and time when the booking was submitted.
updatedAt	Timestamp	-	Optional	Date and time when the booking record was last updated.

Messages

Table 5 provides an overview of the Messages table utilized in the system. This table represents the data component responsible for managing the

communication between users within the NeighborHelp platform. Each record corresponds to a single message and is uniquely identified by messageId.

The table establishes relationships through bookingId, senderId, and receiverId to ensure that each message is properly associated with a specific service transaction and correctly delivered between the interacting users. These fields support real-time communication between customers and service providers within the context of a booking.

It stores essential message content such as messageText for text-based communication and mediaUrl for any attached images or files stored in Firebase Storage. The messageType field categorizes the message as Text, Image, or System, allowing the system to properly handle different forms of communication. Additionally, the isRead field tracks whether the recipient has viewed the message, supporting read-receipt functionality. Finally, the sentAt timestamp records the exact date and time when the message was sent.

Table 5: Data Dictionary for Messages Entity

Field Name	Data Type	Field Length	Constraint	Description
messageId	String	20	Document ID, Unique	Unique identifier of each message document.
bookingId	String	20	Required, Reference	References the bookingId associated with the conversation.
senderId	String	20	Required, Reference	References the userId of the message sender.
receiverId	String	20	Required, Reference	References the userId of the message recipient.

messageText	String	50	Optional	Text content of the message.
mediaUrl	String	100	Optional	URL of an attached image or file stored in Firebase Storage.
messageType	String	100	Required	Type of message, such as Text, Image, or System.
isRead	Boolean	-	Default false	Indicates whether the recipient has read the message.
sentAt	Timestamp	-	Required	Date and time when the message was sent.

Reviews

Table 6 provides an overview of the Reviews table utilized in the system. This table represents the data component responsible for storing and managing customer feedback on completed service transactions within the NeighborHelp platform. Each record corresponds to a single review and is uniquely identified by reviewId.

The table establishes relationships with several key entities in the system through bookingId, customerId, providerId, and serviceId. These fields ensure that each review is properly linked to the corresponding service transaction, the customer who submitted the feedback, the service provider being evaluated, and the specific service rendered.

It stores essential evaluation data such as the rating, which reflects the customer's overall satisfaction on a scale of 1 to 5, and the comment field, which contains optional written feedback. To support sentiment analysis, the table also includes sentimentLabel, which categorizes the review as Positive, Neutral, or Negative, and sentimentScore, which provides a computed numerical value

indicating the intensity of the sentiment expressed. Finally, the createdAt timestamp records when the review was submitted.

Table 6: Data Dictionary for Reviews Entity

Field Name	Data Type	Field Length	Constraint	Description
reviewId	String	20	Document ID, Unique	Unique identifier of each review document.
bookingId	String	20	Required, Reference	References the bookingId of the completed service transaction.
customerId	String	20	Required, Reference	References the userId of the customer who submitted the review.
providerId	String	20	Required, Reference	References the providerId of the reviewed service provider.
serviceId	String	20	Required, Reference	References the serviceId of the reviewed service.
rating	Number	-	Required	Star rating from 1 as the lowest to 5 as the highest.
comment	String	100	Optional	Written feedback provided by the customer.
sentimentLabel	String	20	Optional	Sentiment classification of the review (Positive, Neutral, Negative).
sentimentScore	Float	-	Optional	Computed sentiment score indicating strength of sentiment.
createdAt	Timestamp	-	Required	Date and time when the review was submitted.

Notifications

Table 7 provides an overview of the Notifications table utilized in the system. This table represents the data component responsible for storing and managing system-generated notifications delivered to users within the NeighborHelp platform. Each record corresponds to a single notification and is uniquely identified by notificationId.

The table links each notification to a specific user through userId, ensuring that alerts and updates are delivered to the correct recipient. It stores essential details such as the notification title and message content, which provide users with clear and concise information regarding important activities and events within the system, such as booking updates, messages, or service-related alerts.

Table 7: Data Dictionary for Notifications Entity

Field Name	Data Type	Field Length	Constraint	Description
notificationId	String	20	Document ID, Unique	Unique identifier of each notification document.
userId	String	20	Required, Reference	References the userId of the notification recipient.
title	String	50	Required	Short title of the notification.
message	String	50	Required	Notification content displayed to the user.
notificationType	String	50	Required	Type of notification such as Booking Request, Booking Confirmed, New Message, Appointment Reminder, or Review Request.

relatedBookingId	String	20	Optional, Reference	References the bookingId related to the notification, if applicable.
isRead	Boolean	-	Default false	Indicates whether the notification has been read.
createdAt	Timestamp	-	Required	Date and time when the notification was generated.

Verifications

The table 8 is responsible for storing and managing the identity verification records of service providers within the NeighborHelp system. It supports the integration of the Didit identity verification platform, which is used to validate government-issued identification documents and perform biometric facial verification through selfie matching. This table records essential information such as the type of identification submitted, uploaded ID documents, selfie images, and the Didit session ID used to track each verification process.

In addition, the table maintains both system-generated and admin-managed verification statuses, including the verification result returned by Didit and the final decision made by the administrator. It also logs important timestamps such as the submission date and the review date, ensuring transparency and traceability of the verification process.

Table 8: Data Dictionary of Verification Entity

Field Name	Data Type	Field Length	Constraint	Description
------------	-----------	--------------	------------	-------------

verificationId	String	20	Primary Key, Unique	Unique identifier for each verification record.
providerId	String	20	Foreign Key	References the service provider undergoing verification.
idType	String	50	Required	Type of valid ID submitted (e.g., National ID, Driver's License, Passport).
idDocumentURL	String	255	Required	URL of the uploaded government-issued ID stored in Firebase Storage.
selfieURL	String	255	Required	URL of the selfie image used for facial verification
diditSessionId	String	100	Required, Unique	Session ID generated by Didit for tracking the verification process.
verificationResult	String	20	Optional	Result returned by Didit (e.g., Verified, Failed, Pending)
status	String	20	Required	Current verification status (e.g., Pending, Under Review, Approved, Rejected).

reviewedBy	String	200	Optional	Administrator ID who reviewed the verification request.
reviewedAt	Timestamp	-	Optional	Date and time when the verification was reviewed
submittedAt	Timestamp	-	Required	Date and time when the verification request was submitted.

Technologies, Concepts, and Theories

This section presents the key models, algorithms, and technologies utilized in the development of NeighborHelp: A Community-Based Service Finder and Booking Platform. It explains how data is processed from user input to service matching and booking management, as well as the tools and frameworks used to implement the system. The discussion follows the major stages of the system workflow, including data collection, data pre-processing, handling of missing values, feature extraction, normalization, data categorization, matching, scoring calibration, and recommendation algorithms.

Data Collection

Data collection in the NeighborHelp system is performed through user interactions and administrative processes. Customers provide personal information, service requests, booking details, messages, feedback, ratings, reviews, and reports. Service providers submit provider information, service listings, availability schedules, booking confirmations, messages, and verification documents. The Verification Agency reviews the submitted verification documents and returns verification results and status, while the System Administrator manages

user accounts, monitors bookings and verification status, and generates reports. All collected data are stored in the NeighborHelp Database (Cloud Firestore). The system also utilizes the Google Maps API for location-based services and Firebase Cloud Messaging for real-time notifications.

Data Pre-Processing

Before being used by the system, all collected data undergoes preprocessing to ensure accuracy, consistency, and reliability. Input validation checks required fields, email formats, data types, service categories, GPS coordinates, and booking schedules. Booking requests are validated against provider availability to prevent scheduling conflicts, while duplicate accounts are prevented through Firebase Authentication.

The system also verifies that service providers have completed the required profile information and have been approved by the Verification Agency before appearing in customer search results. Providers must have a verified account, service category, service area, and valid location coordinates to be included in GPS-based matching. This preprocessing helps maintain accurate, consistent, and reliable data throughout the NeighborHelp platform.

Before submitting a review, customers are required to select a star rating that reflects their overall service experience. Based on the selected rating, the system displays a contextual writing guide to assist customers in writing appropriate feedback. For example, customers who select one or two stars are encouraged to describe the problems they encountered or what they did not like about the service, while those who select four or five stars are prompted to share positive experiences and explain what they appreciated or would recommend. This guided review process promotes consistency between numerical ratings and written feedback, improving the clarity, reliability, and usefulness of customer reviews.

Handling Missing Values

Missing values are handled based on whether the data field is required or optional. Required fields such as full name, email address, user role, service category, provider location, booking date, and booking status must be completed before the system accepts the record. If any required value is missing, the system displays a validation message and prevents the transaction from continuing.

Optional fields such as profile photo, biography, service images, and additional customer notes may be left blank without affecting system functionality. These are replaced with default values such as default profile images or empty text placeholders.

For service provider records, only providers with complete profiles, valid GPS coordinates, and approved verification status from the Verification Agency are included in customer search results. Providers with no ratings yet are assigned a global average rating as a temporary value instead of zero. Providers without booking history are treated as neutral in completion rate computation. Providers without GPS coordinates are excluded from location-based matching until data is completed.

Feature Extraction

Feature extraction identifies the most relevant attributes used for service matching and provider ranking. These include GPS-based distance computed using the Haversine formula, Bayesian average rating for provider reliability, availability status, booking completion rate, and sentiment score derived from review text analysis.

The inclusion of sentiment analysis allows the system to capture qualitative feedback expressed in written reviews, where each review is processed and classified as positive, neutral, or negative. These classifications are then converted

into a sentiment score that represents the overall emotional tone of customer feedback. These features were selected because they represent the major factors influencing customer decision-making, including proximity, service quality and reliability, availability, and user satisfaction.

Before storing a customer review, the system compares the predicted sentiment of the review text with the selected star rating. If the detected sentiment is inconsistent with the assigned rating (e.g., a negative review with a 5-star rating), the system displays a validation message prompting the customer to review and correct either the comment or the rating before submission. This validation helps improve the consistency and reliability of customer feedback while preserving the user's ability to revise their input.

Normalization Process

Normalization converts different feature values into a unified scale, typically ranging from 0 to 1, allowing fair combination in the scoring algorithm. For NeighborHelp, the following normalized values are used: Proximity Score, which is derived through an inverse transformation of distance where closer providers receive higher scores; Rating Score, computed by dividing the Bayesian average rating by the maximum rating value of 5; Availability Score, represented as a binary value where available providers are assigned 1 and unavailable providers are assigned 0; Completion Rate Score, calculated as the ratio of completed bookings to accepted bookings; and Sentiment Score, which is a normalized sentiment polarity score derived from review text analysis. Sentiment scores are normalized so that positive feedback contributes higher values while negative feedback reduces the overall score, ensuring that the emotional tone expressed in user reviews is fairly reflected in provider evaluation.

Data Categorization and Status Classification

The NeighborHelp system classifies platform data into meaningful groups to support service discovery, booking management, and analytics. Service listings are categorized by type such as household services, technical repairs, personal services, and support services. Bookings are classified by status including pending, accepted, declined, completed, cancelled, or rescheduled.

Users are classified into roles: customer, service provider, verification agency or administrator. This classification supports role-based access control and ensures appropriate feature access. Provider search results may also be grouped based on relevance tiers generated from the final composite score.

Matching Algorithm Selection

NeighborHelp requires an effective provider matching mechanism to recommend the most suitable service providers based on multiple criteria, including geographic proximity, service quality, availability, verification status, and booking reliability. To identify the most appropriate approach, three matching algorithms were evaluated during the system design phase: Weighted Scoring, Collaborative Filtering, and Neural Collaborative Filtering. Each approach differs in terms of data requirements, computational complexity, explainability, and suitability for newly developed platforms.

Collaborative Filtering is widely used in recommender systems because it predicts user preferences based on historical interactions and similarities among users. Neural Collaborative Filtering further improves recommendation accuracy by employing deep learning techniques to model complex user-item relationships. However, both approaches depend heavily on historical interaction data and are affected by the cold-start problem, where recommendations become unreliable when insufficient user activity data is available [51], [52], [56].

Since NeighborHelp is a newly developed community-based service platform, sufficient historical booking and interaction data are not yet available. Consequently, recommendation methods that rely on user behavior patterns may not provide accurate results during the initial deployment stage. Therefore, the proponents selected the Weighted Scoring Algorithm as the primary provider matching approach. Unlike collaborative filtering techniques, weighted scoring can generate recommendations using predefined criteria such as distance, ratings, availability, verification status, and booking completion rate without requiring historical interaction data [52].

As shown in Table 8, Weighted Scoring demonstrates lower data requirements, reduced computational complexity, higher explainability, and better suitability for new platforms compared to Collaborative Filtering and Neural Collaborative Filtering. These characteristics make it particularly appropriate for community-based service platforms where transparency, fairness, and ease of maintenance are important considerations [52].

Based on this evaluation, Weighted Scoring was selected because it enables immediate deployment, provides transparent and adjustable recommendations, and effectively ranks service providers using normalized criteria such as proximity score, provider rating score, availability score, booking completion rate score, and sentiment score derived from customer reviews. This approach ensures that recommendations remain understandable, fair, and adaptable while maintaining efficient system performance [51].

Table 8: Comparison of Matching Algorithm Approaches

Criteria	Weighted Scoring (Selected)	Collaborative Filtering	Neural Collaborative Filtering
Data Requirement	Low	Medium	High

Historical Data Needed	No	Yes	Yes
Cold Start Problem	Minimal	High	High
Computational Complexity	Low	Medium	High
Explainability	High	Moderate	Low
Ease of Implementation	High	Moderate	Difficult
Maintenance	Easy	Moderate	Complex
Suitability for New Platforms	High	Moderate	Low
Suitability for NeighborHelp	High	Moderate	Low

Haversine Formula for Proximity Calculation

The Haversine formula is the foundational algorithm used by the GPS-based provider matching module to calculate the approximate distance between two geographic coordinates, specifically the customer's selected service location and each registered service provider's stored location. The formula is derived from spherical trigonometry and computes the shortest distance over the Earth's surface between two points defined by their latitude and longitude values [23]. It is suitable for location-based service platforms because it allows the system to estimate how near or far a provider is from a customer.

The formula is expressed as: $a = \sin^2(\Delta\text{lat}/2) + \cos(\text{lat1}) \times \cos(\text{lat2}) \times \sin^2(\Delta\text{lon}/2)$ $c = 2 \times \text{atan2}(\sqrt{a}, \sqrt{1-a})$ $d = R \times c$, where R is the Earth's mean radius of 6,371 kilometers, lat1 and lat2 are the latitudes of the two points in radians, Δlat

is the difference in latitude, Δlon is the difference in longitude, and d is the resulting distance in kilometers [23].

In NeighborHelp, the Haversine computation may be handled through Firebase Cloud Functions or within the Flutter application depending on the implementation design. For better control and consistency, provider filtering and ranking may be processed through Firebase Cloud Functions before the ranked list of nearby providers is returned to the Flutter mobile application. Only providers whose calculated distance falls within the customer's selected service radius are included for further ranking.

Weighted Scoring Algorithm for Provider Ranking

The weighted scoring algorithm assigns a composite relevance score to each provider that passes the proximity filter, enabling the system to rank results in order of overall suitability rather than distance alone. This approach ensures that customers are not always directed to the nearest provider if that provider has a significantly lower rating or is currently unavailable [55].

The composite score is calculated using four features: proximity score, rating score, availability score, and completion rate score. The *Proximity Score* gives importance to nearby providers. The *Rating Score* measures service quality based on customer reviews. The *Availability Score* prioritizes providers who are currently available. The *Completion Rate Score* reflects the provider's reliability in completing accepted bookings. Providers are sorted in descending order of composite score and displayed to the customer in a ranked list together with the map view. This approach is consistent with multi-criteria decision-making because it combines several important criteria into one final score [54].

Recommendation Score Computation

NeighborHelp computes a recommendation score for each eligible service provider using a weighted scoring algorithm to determine the most suitable

providers for a customer's service request. Only providers who have successfully completed the verification process are included in the recommendation process. The recommendation score is calculated by combining the normalized values of four ranking criteria according to their assigned weights using the formula: **Recommendation Score (RS) = (0.40 × P) + (0.30 × R) + (0.20 × A) + (0.10 × C)**, where RS represents the Recommendation Score, P represents the Proximity Score, R represents the Rating Score, A represents the Availability Score, and C represents the Completion Rate Score. The assigned weights reflect the relative importance of each criterion in the recommendation process. Proximity is assigned the highest weight (40%) because NeighborHelp prioritizes nearby service providers to reduce travel distance, transportation costs, and service response time. Rating contributes 30% to represent previous customer satisfaction and service quality, while Availability contributes 20% to prioritize providers who are currently available to accept bookings. Completion Rate contributes the remaining 10% to measure provider reliability based on successfully completed service transactions. After calculating the recommendation scores, all eligible service providers are sorted in descending order, and the highest-ranked providers are presented to customers through the provider recommendation module.

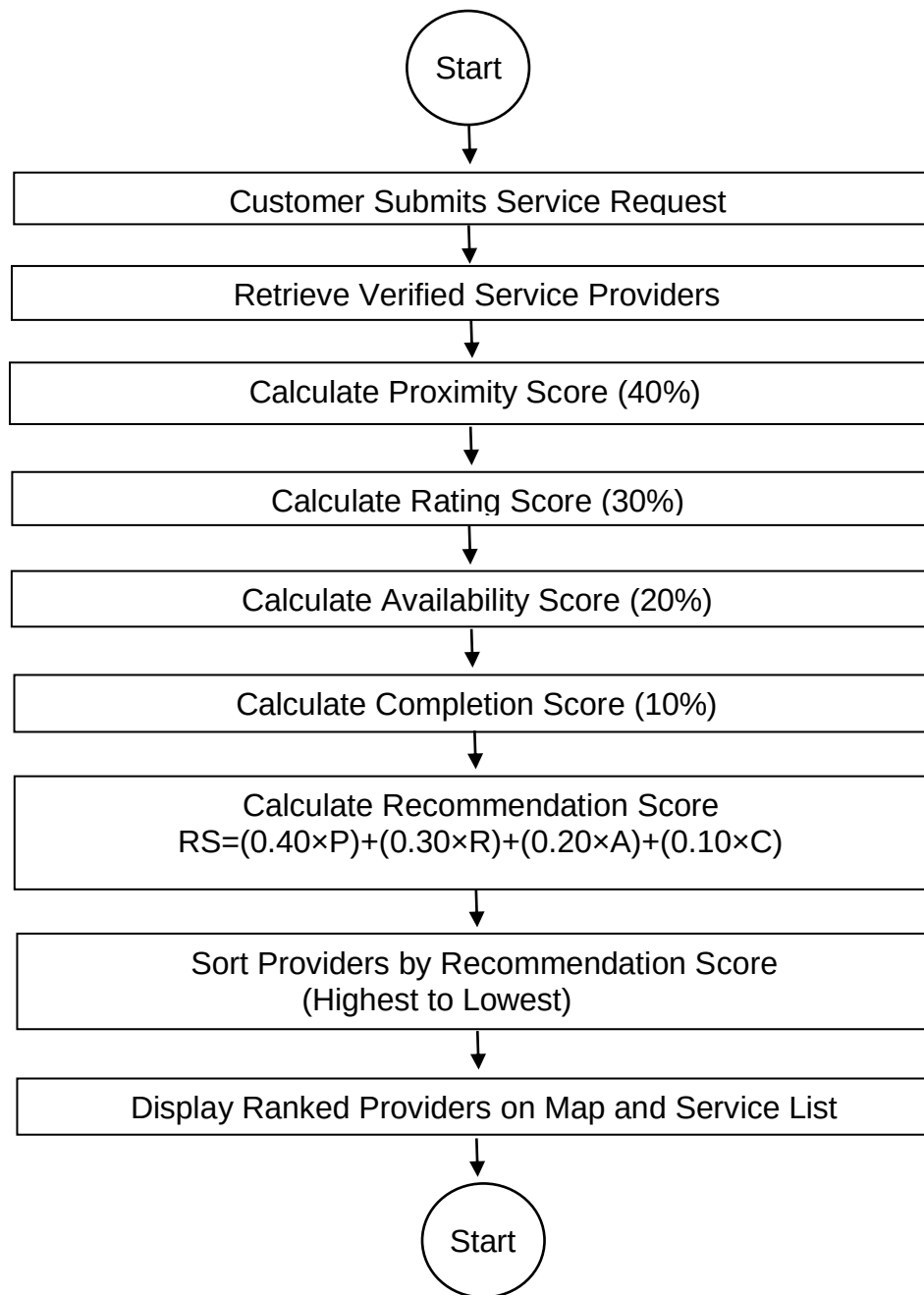


Figure 9: Flowchart of the Weighted Scoring Algorithm for Provider Recommendation

Figure 9 illustrates the workflow of the weighted scoring algorithm used by NeighborHelp. After a customer submits a service request, the system retrieves all verified service providers who satisfy the eligibility requirements. The system then computes the

Proximity Score, Rating Score, Availability Score, and Completion Rate Score for each provider. These values are combined using the weighted scoring formula to obtain the Recommendation Score. Finally, the providers are sorted in descending order of their recommendation scores, and the ranked results are displayed to the customer through the map and service list interface.

Bayesian Average Rating for Review Aggregation

The Bayesian Average formula is used to compute provider ratings in a fair and statistically reliable manner, particularly for providers with a small number of reviews. A simple arithmetic mean may unfairly inflate the reputation of providers who have received only one or two five-star ratings, giving them an advantage over experienced providers with larger but more varied review histories. The Bayesian Average compensates for this by pulling the calculated rating toward the global platform mean when the number of reviews is low [51]. Before a customer rating is included in the computation, the system validates the consistency between the submitted review text and the selected star rating using sentiment analysis. If an inconsistency is detected, the customer is prompted to revise either the review or the rating before submission. This helps ensure that only consistent and reliable feedback contributes to the provider's overall rating.

The formula is expressed as: $\text{Bayesian Average} = (C \times m + \sum r) / (C + n)$, where C is a confidence weight representing the minimum number of reviews needed for a rating to be fully trusted, m is the global mean rating across all providers on the platform, $\sum r$ is the sum of all individual ratings received by the provider, and n is the total number of reviews received. As the provider accumulates more reviews, the formula naturally converges toward the true arithmetic mean, making the adjustment self-correcting over time. This approach is consistent with techniques used in recommender systems and review-based platforms [51], [52].

Scoring Calibration

Scoring calibration refers to the process of reviewing and adjusting the weights used in the provider ranking formula. The initial scoring weights are based on the purpose of NeighborHelp, which is to help customers find nearby, available, and reliable service providers within the community.

Table 9: The Initial Calibration of the Provider Ranking Score

Ranking Factor	Weight	Reason
Proximity Score	40%	Nearby providers are prioritized because the system is community-based and services are performed on-site.
Rating Score	30%	Customer feedback helps measure service quality and provider trustworthiness.
Availability Score	20%	Available providers are more likely to respond to booking requests quickly.
Completion Rate Score	10%	Providers with successful booking history are considered more reliable.

Verification status is treated as a mandatory eligibility requirement rather than a weighted ranking factor. Only service providers who have successfully completed the verification process are included in the ranking algorithm. Once verified, providers are evaluated using the weighted scoring criteria presented in Table 9.

Priority Ranking and Fairness of the Matching Algorithm

The priority ranking in NeighborHelp is based on a weighted scoring approach that evaluates service providers using objective and measurable criteria. The assigned weights were determined according to the primary goal of the platform, which is to recommend providers who can efficiently and reliably deliver services within the customer's community. Proximity Score was assigned the highest weight 40% because the platform prioritizes nearby providers to reduce travel time, transportation costs, and service response time. Rating Score was assigned 30% since customer ratings reflect previous service quality and satisfaction. Availability Score received 20% to prioritize providers who are ready to accept bookings, while Completion Rate Score was assigned 10% because it indicates provider reliability based on successfully completed bookings.

To ensure fairness, the ranking algorithm evaluates all eligible service providers using the same predefined criteria and weighting scheme. The algorithm does not consider personal characteristics such as age, gender, religion, or socioeconomic background. Instead, recommendations are generated solely from objective service-related factors, including geographic proximity, provider ratings, availability, and booking completion rate. Verification status serves as an eligibility requirement, ensuring that only verified providers are included in the recommendation process before the weighted ranking is applied. This standardized evaluation ensures that every provider is assessed consistently and transparently.

To reduce potential ranking bias, NeighborHelp incorporates several fairness mechanisms. Only service providers who have successfully completed the verification process are eligible for inclusion in the recommendation results. Newly registered providers are assigned a neutral initial completion rate and a Bayesian Average rating, which reduces the advantage of providers with only a few reviews by considering both the provider's ratings and the platform-wide average. This approach provides a more statistically reliable assessment of provider performance while preventing newly registered providers from being unfairly disadvantaged. In addition, the ranking weights may be recalibrated based on system testing, user

feedback, and administrator evaluation to maintain balanced recommendations and improve fairness for both new and experienced service providers.

Although the initial weights were selected based on the objectives of the proposed system, they may be recalibrated after system testing, user acceptance evaluation, and administrator feedback to improve recommendation accuracy and maintain fairness among service providers. For example, if users prefer highly rated providers even if they are slightly farther away, the rating weight may be increased. If users value fast response and immediate booking more, the availability weight may be increased. If administrators observe that new providers are not getting enough visibility, the completion rate weight may be adjusted to avoid giving too much advantage to older providers.

The purpose of scoring calibration is to ensure that the ranking result remains practical, fair, and aligned with the expectations of customers, service providers, and administrators.

Possible Bias in Provider Ranking

Although the weighted scoring algorithm improves provider recommendations, certain forms of bias may still occur because recommendations are based on predefined ranking criteria. One possible bias is location bias. Since proximity has the highest weight, providers located closer to customers or within the central service area may appear more frequently in recommendation results. As a result, qualified providers located farther away may receive fewer booking opportunities despite offering comparable service quality.

Another potential bias is review bias. Providers with many positive customer reviews may consistently rank higher than newly registered providers who have not yet accumulated sufficient ratings or completed bookings. This may limit the visibility of new providers during their initial participation on the platform.

In addition, availability bias may occur when providers who frequently indicate that they are available are recommended more often than equally qualified

providers with limited schedules. Similarly, popularity bias may arise when providers with an established history of successful bookings continue to receive more recommendations, allowing them to accumulate additional ratings and completed transactions over time.

While these biases cannot be completely eliminated, NeighborHelp incorporates fairness mechanisms to reduce their impact. These include treating verification as an eligibility criterion, applying Bayesian Average ratings, assigning neutral initial values to newly registered providers, and allowing ranking weights to be recalibrated based on system evaluation and user feedback. Furthermore, administrators may monitor provider visibility through the analytics dashboard to identify potential imbalances and support future improvements to the recommendation process.

Security and Risk Scenario Analysis

NeighborHelp incorporates several security measures to reduce potential risks associated with provider registration, service transactions, and user interactions. Although the system cannot guarantee that every provider or customer will always act appropriately, it implements preventive and corrective mechanisms to minimize fraudulent activities, inappropriate content, and misuse of the platform.

Before a service provider becomes eligible to offer services, the provider must successfully complete the identity verification process through the integrated third-party verification agency. Verified providers are then required to complete their service profiles, which undergo an administrative review to ensure that the submitted information complies with the platform's content policies and service standards before becoming visible to customers.

In addition, the system supports post-transaction monitoring through ratings, reviews, and user reports. Customers may report incidents such as fraudulent behavior, harassment, theft, misleading service information, or other policy violations. Upon receiving a report, the administrator reviews the submitted

evidence and may place the provider under Pending Investigation while the complaint is evaluated. Depending on the investigation outcome, the provider's account may remain active, be temporarily suspended, or be permanently banned for repeated or serious violations of the platform's policies.

Table 10: Security and Risk Scenario Analysis

Scenario	Risk	Mitigation
Fake provider registration	Fraud	Third-party identity verification
Fake service listing	Misleading information	Administrative service profile review
Fraudulent booking	Service abuse	Booking history, reporting, suspension
Malicious customer	Provider harassment	Complaint handling and account suspension
False reviews	Reputation manipulation	Verified-booking-only review submission

Table 10 presents representative security and risk scenarios considered in the design of Neighborhelp. The platform implements multiple preventive and corrective mechanisms to reduce security risks throughout the service transaction process. Identity verification performed through the integrated third-party verification agency minimizes the risk of fake provider registrations, while administrative review of provider service profiles helps prevent misleading or inappropriate information from and complaint handling procedures to detect fraudulent transactions and inappropriate user behavior. Furthermore, only customers who have completed verified service transactions are permitted to submit ratings and reviews, reducing the likelihood of false reviews and reputation manipulation. Together, these security controls improve platform reliability, strengthen user trust, and support safer community-based service transactions.

Sentiment Analysis Integration

Sentiment analysis is integrated into NeighborHelp to extract meaningful insights from user-generated reviews that cannot be fully represented by numerical ratings alone. After a completed service transaction, customers are encouraged to submit both a numerical rating and a textual review describing their service experience. The textual feedback is then processed by the sentiment analysis module to identify the overall opinion expressed by the customer.

Before classification, each review undergoes preprocessing to improve data quality by removing unnecessary characters, converting text to a consistent format, and preparing the content for analysis. The processed review is then classified into one of three sentiment categories: positive, neutral, or negative. The resulting sentiment label is stored together with the customer's review and numerical rating in the system database.

The classified sentiment is integrated into the provider analytics module, where it contributes to reputation indicators, review summaries, and service performance reports. Unlike the provider matching algorithm, which recommends service providers using objective criteria such as geographic proximity, provider ratings, availability, and completion rate, sentiment analysis is primarily used to support service quality monitoring and administrative decision-making. Administrators can analyze customer feedback trends, identify recurring service issues, and monitor overall provider performance through the analytics dashboard.

By combining guided review submission with sentiment analysis, NeighborHelp improves the consistency, clarity, and reliability of customer feedback while providing meaningful insights that support service quality monitoring and continuous platform improvement.

Sentiment Analysis Algorithm Evaluation

To determine the most suitable sentiment analysis technique for NeighborHelp, three machine learning algorithms were evaluated: Naïve Bayes, Support Vector Machine (SVM), and Long Short-Term Memory (LSTM). The dataset used in this study was the Yelp Open Dataset, specifically the `yelp_academic_dataset_review.json` file, which contains user-generated business reviews. The original Yelp Open Dataset contains approximately 6.99 million reviews. However, due to processing limitations and to make the training manageable in Google Colab, the researchers loaded only 1,000,000 review records for the experiment.

From this loaded dataset, the text review and star rating columns were used for sentiment classification. The star ratings were converted into three sentiment categories: negative, neutral, and positive. Reviews with ratings of 1 to 2 stars were labeled as negative, reviews with 3 stars were labeled as neutral, and reviews with 4 to 5 stars were labeled as positive. After converting the ratings into sentiment labels, the class distribution showed that there were 680,509 positive reviews, 216,537 negative reviews, and 102,954 neutral reviews. Since the neutral class had the lowest number of samples, the researchers used 102,000 samples per class to balance the dataset. The balancing process was done through under sampling. This means that the researchers selected the same number of reviews from each sentiment category to prevent the models from becoming biased toward the majority class. A total of 102,000 negative reviews, 102,000 neutral reviews, and 102,000 positive reviews were selected.

As a result, the final balanced dataset contained 306,000 reviews. This balanced dataset helped ensure that the models learned equally from the three sentiment classes. Before training the models, text cleaning was applied to the review text. The review text was converted into lowercase, URLs were removed,

non-letter characters were removed, and extra spaces were cleaned. This preprocessing step was necessary to reduce noise in the dataset and prepare the text for machine learning. After cleaning, the dataset was divided using an 80:20 train-test split. This means that 244,800 reviews were used for training, while 61,200 reviews were used for testing. Stratified splitting was applied so that the three sentiment classes remained equally represented in both the training and testing datasets.

Three sentiment analysis models were trained and compared in this study: Naïve Bayes, Support Vector Machine, and Long Short-Term Memory. The Naïve Bayes and Support Vector Machine models used TF-IDF vectorization with a maximum of 50,000 features and unigram-to-bigram word patterns. Naïve Bayes was used because it is a common baseline algorithm for text classification and works by calculating the probability of a class based on word occurrence. Support Vector Machine was used because it is effective in handling high-dimensional text features and can separate classes using a decision boundary. The LSTM model was used because it is a deep learning model that can learn word sequence and context, which is useful in understanding the meaning of review sentences. The LSTM model used text vectorization with 30,000 maximum tokens, a sequence length of 150, an embedding dimension of 128, and a bidirectional LSTM layer. Dropout layers were also used to reduce overfitting.

The model was trained using the Adam optimizer, sparse categorical cross entropy loss function, early stopping, and learning rate reduction. Although the maximum number of epochs was set to 25, the training stopped after 4 epochs based on the validation loss behavior. The models were evaluated using the testing dataset composed of 61,200 reviews. The evaluation metrics used were accuracy, precision, recall, and F1-score.

The results showed that Naïve Bayes achieved an accuracy of 73.91%, precision of 74.58%, recall of 73.91%, and F1-score of 74.14%. Support Vector Machine achieved an accuracy of 76.13%, precision of 75.92%, recall of 76.13%,

and F1-score of 76.00%. The LSTM model achieved the highest performance, with an accuracy of 77.27%, precision of 77.13%, recall of 77.27%, and F1-score of 77.18%. Based on the results, LSTM performed the best among the three models. This indicates that the LSTM model was better at classifying review sentiments because it can learn patterns from the sequence and context of words. However, it also required the longest training time, with 8,988.38 seconds, compared to 85.25 seconds for Naïve Bayes and 128.26 seconds for Support Vector Machine.

Therefore, although LSTM achieved the highest accuracy and F1-score, Support Vector Machine may still be considered a practical alternative because it produced competitive results with a much shorter training time. The formulas used to compute the evaluation metrics are as follows: $\text{Accuracy} = (TP + TN) / (TP + TN + FP + FN)$ $\text{Precision} = TP / (TP + FP)$ $\text{Recall} = TP / (TP + FN)$ $\text{F1-score} = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$ In these formulas, TP means True Positive, TN means True Negative, FP means False Positive, and FN means False Negative. True Positive refers to reviews correctly classified under their actual sentiment class. True Negative refers to reviews correctly identified as not belonging to a certain class. False Positive refers to reviews incorrectly predicted as belonging to a sentiment class. False Negative refers to reviews that actually belong to a sentiment class but were incorrectly classified as another class.

Accuracy measures the overall correctness of the model by determining how many reviews were classified correctly out of all testing samples. Precision measures how reliable the model is when it predicts a review as negative, neutral, or positive. Recall measures how well the model detects all actual reviews belonging to a sentiment class. F1-score combines precision and recall into one balanced score, making it useful when comparing the overall performance of classification models. In this study, the reported precision, recall, and F1-score were computed using the weighted average. This means that the metric was first computed per class, then averaged based on the number of actual samples per class. Since the testing set was balanced with 20,400 reviews per class, the

weighted average gave a fair overall summary of model performance across negative, neutral, and positive sentiments.

The trained models were also tested using sample NeighborHelp-related reviews. This was done to demonstrate how the sentiment analysis model can be applied to the proposed system. Through this feature, NeighborHelp can analyze customer feedback and classify reviews as negative, neutral, or positive. The result can support the system's feedback, rating, provider reputation, and analytics features by helping administrators understand user satisfaction and service quality.

Table 10: Performance Comparison of Sentiment Analysis Algorithms

Algorithm	Accuracy	Precision	Recall	F1-Score
Naïve Bayes	73.41	73.99	73.41	73.62
Support Vector Machine (SVM)	74.82	74.60	74.82	74.69
Long Short- Term Memory (LSTM)	76.22	76.23	76.22	76.22

Technologies Used in the System

NeighborHelp is built using a combination of modern mobile, web, cloud database, authentication, storage, notification, and mapping technologies that work together to support service discovery, GPS-based provider matching, realtime messaging, booking management, ratings and reviews, and administrative analytics. The selection of these technologies was based on their reliability,

ecosystem maturity, community support, ease of integration, and suitability for a cross-platform community service booking platform.

Flutter

Flutter is the primary development framework used to build the NeighborHelp mobile application and web-based administrator dashboard. It allows the proponents to develop applications for multiple platforms using a single codebase, making it suitable for a system that requires both mobile access for customers and service providers and web access for administrators. Through Flutter, the system can provide a consistent user interface, responsive layouts, and interactive screens for service browsing, provider profiles, booking requests, inapp messaging, ratings and reviews, notifications, and dashboard monitoring.

Flutter was selected because it supports fast development through reusable widgets and a flexible user interface structure. In NeighborHelp, Flutter handles the customer-facing and provider-facing mobile application as well as the administrator web dashboard. This allows the proponents to maintain a unified development environment and reduce the complexity of managing separate frameworks for mobile and web platforms.



Figure 10: Flutter

Dart

Dart is the programming language used with Flutter in developing the NeighborHelp application. It is used to implement the logic of the mobile and web interfaces, including screen navigation, form validation, data handling, provider search results, booking workflows, and interaction with Firebase services. Dart

supports object-oriented programming, which allows the proponents to organize the system into reusable classes and components such as user models, service models, booking models, review models, and notification models.

In NeighborHelp, Dart helps manage client-side operations such as capturing form inputs, displaying provider listings, sending booking requests, loading chat messages, and showing real-time updates from the database. Its integration with Flutter makes the development process more organized and easier to maintain.



Figure 11: Dart

Firestore Authentication

Firestore Authentication manages user account creation, secure login, and session handling for the NeighborHelp platform. It supports email and password authentication for customers, service providers, and administrators. The system uses Firestore Authentication to verify user identity before allowing access to protected features such as booking services, managing service listings, sending messages, submitting reviews, and accessing the administrator dashboard.

Role-based access control is implemented by associating each authenticated user with a specific role stored in Cloud Firestore, such as Customer, Service Provider, or Administrator. This ensures that users can only access the features appropriate to their role. For example, customers can search and book services, service providers can manage listings and respond to booking requests, and administrators can manage accounts, monitor platform activity, and view reports.



Figure 12: Firebase Authentication

Cloud Firestore

Cloud Firestore serves as the main cloud database of the NeighborHelp system. It stores structured platform records such as user profiles, service provider profiles, service listings, booking transactions, messages, reviews, notifications, and activity logs. Firestore was selected because it supports real-time data synchronization, which is important for features such as in-app messaging, booking status updates, notification records, and live administrative monitoring.

In NeighborHelp, Firestore allows updates to be reflected immediately across connected devices. For example, when a customer submits a booking request, the provider can receive the updated booking record in real time. When a provider accepts or declines a booking, the customer can immediately see the updated status. The same real-time capability supports messaging between customers and providers, making communication faster and more organized within the platform.



Figure 13: Cloud Firestore

Firestore Storage

Firestore Storage is used to store media files associated with the NeighborHelp platform. These files may include service provider profile photos, service listing images, verification documents, and image attachments sent through in-app messaging. Instead of storing large files directly in the database, the system stores the files in Firestore Storage and saves their download URLs or references in Cloud Firestore.

This approach improves system organization and performance because text-based records and media files are managed separately. It also supports secure file access because Firestore Storage can be configured with access rules based on authenticated users and their assigned roles.



Figure 14: Firestore Storage

Firestore Cloud Messaging

Firestore Cloud Messaging is used to deliver push notifications and in-app alerts to customers, service providers, and administrators. Notifications are important in NeighborHelp because users need timely updates about booking requests, booking confirmations, booking cancellations, incoming messages, appointment reminders, and review requests.

When a booking-related event occurs, such as a new booking request or a provider accepting a customer's request, the system can trigger a notification to the affected user. This helps ensure that customers and providers remain updated even when they are not actively using the application. Firestore Cloud

Messaging improves communication efficiency and supports a smoother booking experience.



Figure 15: Firebase Cloud Messaging

Firebase Cloud Functions

Firebase Cloud Functions is used to perform secure backend operations that should not be handled directly on the client side. These operations may include booking validation, notification triggering, provider ranking, review rating recalculation, administrative checks, and other server-side processes. By using Cloud Functions, NeighborHelp can execute important logic in a controlled backend environment while still maintaining a Firebase-centered architecture.

For example, when a booking request is created, a Cloud Function may check the provider's availability, update the booking record, and trigger a notification to the provider. When a review is submitted, another Cloud Function may update the provider's average rating. This helps protect the integrity of system processes and reduces the risk of users manipulating important data from the client application.



Figure 16: Firebase Cloud Function

Google Maps Platform

The Google Maps Platform is used to support the location-based features of NeighborHelp. It enables map visualization, location selection, address lookup, and GPS-based service discovery. Through Google Maps integration, customers can search for nearby providers, view service provider locations on a map, and identify providers within a selected service area.

In NeighborHelp, GPS coordinates from customers and service providers are used to support proximity-based matching. The system may use the Haversine formula to calculate the approximate distance between the customer's location and the provider's registered service area. This allows the system to filter and rank providers based on location relevance, making it easier for customers to find nearby cleaners, tutors, plumbers, electricians, repair workers, caregivers, and other local service providers.



Figure 17: Google Maps Platform

Flutter Google Maps Package

The Flutter Google Maps package is used to integrate Google Maps directly into the Flutter application. It allows the system to display interactive maps, provider markers, customer location pins, and service area references within the mobile and web interface. This package helps make the location-based features of NeighborHelp easier to use because customers can visually identify nearby providers instead of relying only on text-based search results.

The map interface supports the service discovery process by showing provider locations and allowing users to understand distance and availability more clearly. It also improves the overall user experience by making the GPS-based matching feature more visual and intuitive.



Figure 18: Flutter Google Maps Package

Analytics and Dashboard Visualization

For the administrator dashboard, NeighborHelp uses Flutter-compatible charting and visualization tools to present system analytics. These tools may include packages such as `fl_chart` or other Flutter chart libraries that can display line charts, bar charts, pie charts, and summary cards. These visualizations help administrators monitor platform activity and service demand.

The dashboard may display reports such as total bookings, most requested service categories, number of active providers, completed bookings, cancelled bookings, average provider ratings, and service demand by location. These analytics support data-driven monitoring and help administrators understand how the platform is being used by the community.

REST APIs for Future Integration

REST APIs may be used for future integrations when the system needs to communicate with external services outside the Firebase environment. Although NeighborHelp primarily uses Firebase services for authentication, database

management, storage, notifications, and backend functions, REST APIs remain useful for possible future enhancements such as payment gateways, SMS services, identity verification tools, or third-party reporting systems.

In the current scope, Firebase services handle most backend communication directly through Flutter packages and SDKs. However, REST API support gives NeighborHelp flexibility for future expansion if additional services are integrated into the platform.



Figure 19: Rest API

Didit Identify Verification

Didit is a cloud-based identity verification platform used to verify the authenticity of users through document validation and biometric face verification. In the NeighborHelp system, Didit is integrated to help verify users during registration by checking government-issued identification documents and matching them with facial recognition scans. This verification process helps reduce fraudulent accounts, increases platform security, and builds trust between customers and service providers. Didit supports automated Know Your Customer (KYC) verification, liveness detection, and identity authentication through a secure API, enabling NeighborHelp to ensure that registered service providers are legitimate before they are allowed to offer services on the platform.



Figure 20: Didit Identify Verification

SYSTEM TESTING AND IMPLEMENTATION

This section presents the testing and implementation procedures of the NeighborHelp system. It describes how the proponents validate the functionality, reliability, usability, security, and performance of the system before deployment. It also explains how the system is developed, tested, refined, configured, and prepared for actual use. The discussion follows the iterative Agile sprint cycles defined in the methodology, ensuring that each module is properly tested and improved before being integrated into the complete platform.

The NeighborHelp system undergoes a structured and iterative testing process to ensure that all functional and non-functional requirements are properly met. Testing activities are performed throughout the four Agile sprints of the implementation phase, allowing the proponents to detect, document, and resolve issues progressively while the system is being developed. The testing strategy combines unit testing, integration testing, system testing, and user acceptance testing to confirm that each module operates correctly both independently and as part of the complete system.

Each test cycle involves the preparation of test cases based on the functional requirements of the system, the execution of these test cases by the proponents, and the documentation of test results. Defects identified during testing are logged, prioritized, and corrected within the same sprint or carried over to the next iteration when necessary. The testing environment uses sample user accounts, dummy GPS coordinates, sample service listings, booking records, message records, ratings, and notification records to simulate real-world service transactions. The proponents may also coordinate with selected community members and local service providers to perform usability testing and user acceptance testing, ensuring that the system aligns with actual service access and booking workflows within the community.

System Test Plan

The system test plan is prepared to verify whether the major modules of NeighborHelp perform according to the expected functional requirements. Each test case includes a specific test scenario and expected result. The expected result serves as the basis for determining whether the feature has passed or requires further revision.

Table 12: System Test Plan

Test Area	Test Scenario	Expected Result
User Registration	A customer or service provider registers using complete and valid information.	The account is successfully created in Firebase Authentication and the user profile is stored in Cloud Firestore.
Login and Authentication	A registered user enters a valid email address and password.	The user is successfully logged in and redirected to the correct interface based on role.
Invalid Login	A user enters an incorrect password or unregistered email address.	The system rejects the login attempt and displays an appropriate error message.
Password Recovery	A user requests a password reset using a registered email address.	The system sends a password reset link through Firebase Authentication.
Role-Based Access Control	A customer attempts to access administrator only features.	The system blocks access and displays only the features allowed for the customer role.
Profile Management	A user updates profile details such as name, contact number, address, or profile photo.	The updated information is saved in Cloud Firestore and reflected in the user profile.

Service Listing Management	A service provider creates, updates, activates, or deactivates a service listing.	The service listing is correctly saved, updated, displayed, hidden, or reactivated based on the provider's action.
GPS-Based Provider Matching	A customer searches for providers within a selected service radius.	The system displays providers within the selected radius and ranks them based on proximity, rating, availability, and completion rate.
Booking Request	A customer submits a booking request with service, date, time, location, and instructions.	The booking request is saved in Cloud Firestore with a pending status and the provider receives a notification.
Booking Response	A service provider accepts or declines a booking request.	The booking status is updated in real time and the customer receives the corresponding notification.
In-App Messaging	A customer and service provider exchange messages related to a booking.	Messages are stored in Cloud Firestore and displayed in the correct conversation thread.
Notification Delivery	A booking update, message, or review prompt is triggered.	The correct user receives the notification through Firebase Cloud Messaging or in-app alert.
Ratings and Reviews	A customer submits a rating and review after a completed booking.	The review is stored in Cloud Firestore and the provider's rating is updated.
Analytics Dashboard	The administrator views booking, user, provider, and service reports.	The dashboard displays accurate summary cards, charts, and report data.

Audit Trail and Admin Logs	An administrator verifies a provider or changes an account status.	The action is recorded in the audit log with the date, time, user, and activity details.
Data Export	The administrator exports selected system records.	The selected data is exported in a structured format such as CSV or JSON.
Offline Handling	A user temporarily loses internet connection while using the application.	Previously loaded data remains viewable where applicable, and updates synchronize once the connection is restored.
Security Rules	A user attempts to access records outside their role or ownership.	The system denies unauthorized access based on Firebase security rules.

Sprint 1: User Registration and Authentication

The first sprint focuses on building the foundational components of the NeighborHelp system, particularly user registration, login, profile management, password recovery, account status handling, and role-based access control. The proponents configure Firebase Authentication to support secure email and password authentication for Customers, Service Providers, and System Administrators. User role information, account status, and profile details are stored in Cloud Firestore to determine what features each user can access.

The Flutter mobile application provides registration and login screens for customers and service providers, while the Flutter Web administrative dashboard provides secure access for system administrators. During this sprint, the basic user profile module is created, allowing users to set up and update personal information, contact details, and profile photos stored in Firebase Storage.

Testing during Sprint 1 covers registration, login, logout, password validation, duplicate account detection, password recovery, account lockout handling, role-based access restrictions, profile updates, and Firebase

Authentication session handling. Input validation is also tested to ensure that required fields are completed and invalid data is not accepted. By the end of this sprint, the system provides a working authentication and profile management foundation ready for integration with service discovery features.

Sprint 2: Service Listing and GPS-Based Provider Matching

The second sprint focuses on developing the service listing module and the GPS-based provider matching module, which together form the core service discovery feature of the platform. Service providers are able to create, update, activate, or deactivate their service listings, including category, description, estimated price, availability schedule, and service images. These records are stored in Cloud Firestore, while service-related images are stored in Firebase Storage.

GPS coordinates may be captured from the provider's device during profile setup or selected through the map interface. The Google Maps Platform and Flutter Google Maps package are integrated into the Flutter mobile application to display interactive maps, customer location pins, and provider markers. Address lookup and map visualization support customers in identifying nearby service providers.

The Haversine formula and weighted scoring algorithm are tested using sample customer locations and provider records at different distances and rating levels. Testing in Sprint 2 focuses on verifying that distance calculations are reasonable, search radius filtering works correctly, and providers are ranked based on the selected criteria such as proximity, service category, availability, rating, and booking completion rate. By the end of this sprint, the system supports service listing management and GPS-based provider discovery.

Sprint 3: Booking Management and In-App Messaging

The third sprint focuses on developing the booking and scheduling module and the in-app messaging module. Customers are able to submit booking requests

by selecting a service, preferred date and time, service location, and additional instructions. Service providers receive booking requests and may accept or decline them through the Flutter mobile application. Both customers and providers receive real-time updates when the booking status changes.

Cloud Firestore is used to store booking records and synchronize booking status updates across connected devices. Firebase Cloud Messaging is used to notify users about booking-related events such as new requests, confirmations, cancellations, and appointment reminders. The booking history module also allows customers and providers to view current and past transactions.

The in-app messaging feature is also developed during this sprint. A message thread is linked to a specific booking, allowing the customer and service provider to communicate about service details. Messages are synchronized in real time using Cloud Firestore, while image attachments are stored in Firebase Storage. Testing during Sprint 3 validates the complete booking lifecycle, message sending and receiving, notification delivery, real-time data updates, and booking-related edge cases such as declined bookings, cancelled bookings, rescheduled bookings, and poor network conditions.

Sprint 4: Ratings, Reviews, Analytics, and Notifications

The fourth and final sprint integrates the rating and review module, the analytics dashboard, audit trail, admin activity logging, and the complete notification workflow. After a booking is marked as completed, the customer receives a review request and may submit a star rating and written feedback.

Submitted reviews are stored in Cloud Firestore and displayed on the provider's profile page to support transparency and customer decision-making.

Provider ratings may be calculated using the Bayesian Average formula to produce a fairer rating, especially for providers with fewer reviews. The updated rating may be stored in the provider record and used as one of the factors in the weighted scoring algorithm for future service searches.

The administrative analytics dashboard is developed through Flutter Web using Flutter-compatible charting tools. The dashboard displays platform metrics such as total bookings, most requested service categories, active users, top-rated providers, completed and cancelled bookings, average ratings, and service demand by location. Firebase Cloud Messaging is also finalized to ensure that booking updates, message alerts, appointment reminders, and review prompts are delivered properly.

End-to-end system testing is conducted at the close of this sprint to verify that all modules from previous sprints work together as one complete system. User acceptance testing is then conducted with selected users to confirm that the NeighborHelp system meets the service access, booking, communication, and monitoring needs of the target community.

Implementation Plan

After successful completion of all sprint cycles and resolution of identified issues, the NeighborHelp system is prepared for deployment. Implementation involves configuring Firebase services for production use, deploying the Flutter Web administrative dashboard, preparing the Flutter mobile application for installation on supported Android devices, and conducting user orientation for customers, service providers, and administrators.

The deployment procedure begins with the preparation of the production environment. The proponents create or finalize the Firebase project, configure project settings, connect the Flutter mobile application and Flutter Web dashboard to Firebase, and verify that all required services are enabled. These services include Firebase Authentication, Cloud Firestore, Firebase Storage, Firebase Cloud Messaging, and Firebase Cloud Functions. Google Maps Platform is also configured by preparing the required API key and enabling the necessary mapping and location services.

The hosting setup focuses on making the administrator dashboard accessible through a secure web environment. The Flutter Web dashboard may be deployed through Firebase Hosting or another approved hosting service. Before deployment, the proponents build the Flutter Web application, upload the build files to the hosting environment, configure the hosting domain, and test whether the dashboard can be accessed properly by the administrator. The mobile application may be installed on supported Android devices through an APK file or other approved deployment method during pilot testing.

The Firebase configuration workflow includes enabling email and password authentication, creating the required Cloud Firestore collections, configuring Firestore security rules, setting up Firebase Storage folders and rules, preparing Firebase Cloud Messaging for notification delivery, and deploying Firebase Cloud Functions for backend operations. The proponents also verify that the Flutter application can connect properly to the Firebase project using the correct configuration files and API keys. Test accounts for customers, service providers, and administrators are created to confirm that each role can access the proper features.

Administrator onboarding is conducted before the system is turned over for pilot use. The administrator is oriented on how to log in to the Flutter Web dashboard, manage user accounts, verify service providers, monitor service listings, review booking activities, handle reports or concerns, view analytics, check audit logs, export data, and manage notifications. The administrator is also informed about basic security practices, such as protecting login credentials, reviewing suspicious activities, and reporting system issues to the proponents.

User Acceptance Testing

User Acceptance Testing is conducted to determine whether the completed NeighborHelp system satisfies the needs and expectations of its intended users. The respondents may include selected customers, service providers, system administrators, and other stakeholders who are familiar with the service booking

process. These respondents are asked to use the system and evaluate its quality based on actual interaction with the mobile application and web dashboard.

The evaluation criteria are based on the ISO/IEC 25010 software quality model. The questionnaire may assess functional suitability, usability, reliability, performance efficiency, security, maintainability, and portability. Respondents rate each criterion using a five-point Likert scale.

Table 13: User Acceptance Testing Criteria

Scale	Rating	Verbal Interpretation
5	4.21 – 5.00	Very High
4	3.41 – 4.20	High
3	2.61 – 3.40	Moderate
2	1.81 – 2.60	Low
1	1.00 – 1.80	Very Low

The acceptance summary presents the results of the user acceptance testing. If the system receives acceptable ratings across the ISO/IEC 25010 criteria and only minor issues are identified, the system may be considered ready for deployment or pilot implementation. If major issues are found, the proponents revise the affected modules, conduct another round of testing, and update the documentation before final deployment. This process ensures that the final system is not only functional but also acceptable to the users who will interact with it..

System Security Plan

The System Security Plan of NeighborHelp is developed in alignment with ISO/IEC 27001 security principles. Since the NeighborHelp platform handles sensitive personal information of community members and local service providers,

including user credentials, contact details, GPS location data, booking records, reviews, and communication logs, ensuring the confidentiality, integrity, and availability of this information is a top priority.

Access control measures are enforced through Firebase Authentication, Cloud Firestore security rules, Firebase Storage security rules, and role-based access control. Firebase Authentication verifies user identity before allowing access to protected system features, while Cloud Firestore stores user roles such as Customer, Service Provider, and System Administrator. These roles determine the functions and records each user can access.

All data transmitted between the Flutter mobile application, Flutter Web administrative dashboard, Firebase services, and Google Maps Platform uses secure communication protocols such as HTTPS and Transport Layer Security. Sensitive records stored in Cloud Firestore and media files stored in Firebase Storage are protected through configured security rules. Important system activities such as account updates, booking changes, review submissions, provider verification, data export, and administrative actions may also be logged to support accountability and auditability.

The system also follows the principles of the Philippine Data Privacy Act of 2012 by limiting access to personal and location-based information, protecting user records from unauthorized access, and ensuring that data is used only for legitimate platform functions. Regular review of access permissions, security rules, and backup settings helps maintain the security and reliability of the NeighborHelp platform.

Password Policy

NeighborHelp shall implement a password policy to improve the security of user accounts. Users shall be required to create passwords that meet minimum security requirements, such as an acceptable length and a combination of letters,

numbers, or symbols. Weak or easily guessed passwords should be discouraged to reduce the risk of unauthorized account access.

The system shall also support password recovery through Firebase Authentication. If a user forgets their password, the system may send a password reset link to the registered email address. This ensures that password recovery is handled securely and only through the verified account email.

Session Time-Out Policy

A session time-out policy shall be implemented to reduce the risk of unauthorized access when a user leaves the application or dashboard inactive. For regular users, the application may require re-authentication after extended inactivity or when the authentication session becomes invalid. For the administrator dashboard, stricter session control should be applied because it contains sensitive records, reports, provider verification features, account management tools, and audit logs.

This control helps prevent unauthorized users from accessing active sessions on shared or unattended devices. It also supports better protection of system records and administrative functions.

Brute Force Prevention

The system shall include brute force prevention measures to reduce repeated unauthorized login attempts. When a user enters incorrect login credentials multiple times, the system may temporarily restrict additional login attempts, display a security warning, or rely on Firebase Authentication protections to limit suspicious activities.

Login attempts and suspicious account activities may also be recorded in the system logs when applicable. These records can help the administrator and

proponents monitor possible security threats, identify repeated failed login attempts, and respond to suspicious behavior.

Data Retention Policy

NeighborHelp shall follow a data retention policy to ensure that system records are kept only for legitimate and necessary purposes. User profiles, provider records, service listings, booking transactions, messages, reviews, notifications, reports, and audit logs shall be retained while they are needed for system operation, academic evaluation, reporting, dispute resolution, or legal compliance.

Inactive, outdated, duplicate, or unnecessary records may be archived, anonymized, or deleted based on administrator approval and privacy requirements. Location data and personal information shall be handled carefully because they involve sensitive user information. This policy supports compliance with the Philippine Data Privacy Act of 2012 and promotes responsible data management.

Table 14: Updated System Security Plan Table

ISO/IEC 27001 Control Domain	Security Control	Implementation in NeighborHelp
A.5 Information Security Policies	Security Policy	A set of policies governing secure data handling, user privacy, password management, account access, data retention, and incident response is documented and followed across platform operations.

A.6 Organization of Information Security	Roles and Responsibilities	The roles of Customer, Service Provider, and System Administrator are clearly defined, with specific access rights enforced through Firebase Authentication and role-based access control.
A.8 Asset Management	Classification of Information Assets	User profiles, GPS coordinates, service

		listings, booking records, reviews, messages, notification records, reports, and audit logs are treated as protected system data.
A.9 Access Control	User Access Management	Firebase Authentication verifies user identity, while Cloud Firestore security rules restrict access to records based on the user's assigned role and ownership of data.
A.9 Access Control	Password and Session Management	Password requirements, password reset workflow, and session time-out controls are applied to reduce unauthorized account access.

A.10 Cryptography	Data Encryption	Communication between the Flutter application, Flutter Web dashboard, Firebase services, and Google Maps Platform is protected through HTTPS and TLS. Firebase also provides secure storage mechanisms for database and file records.
A.12 Operations Security	Logging and Monitoring	Important user actions such as logins, booking transactions, profile updates, review submissions, provider verification, data export, and administrative changes may be logged to support accountability and monitoring.
A.13 Communications Security	Secure Data Communication	Data exchanged between the application and Firebase services is transmitted through secure connections, while notification delivery is handled through Firebase Cloud Messaging.
A.14 System Development Security	Secure Development Practices	Input validation, access control checks, Firebase security rules, proper error handling, and protection against repeated failed login attempts are applied to reduce security risks.

A.16 Incident Management	Incident Response	An incident response process is prepared to handle security concerns such as suspicious accounts, unauthorized access attempts, reported misuse, data-related issues, and possible brute force attempts.
A.17 Business Continuity	Backup and Recovery	Cloud Firestore and Firebase Storage backup and recovery configurations are reviewed to reduce the risk of data loss and support system continuity.
A.18 Compliance	Legal and Regulatory Compliance	The system follows the principles of the Philippine Data Privacy Act of 2012 and applies relevant ISO/IEC 27001 practices for handling personal and locationbased data.

System Maintenance Plan

The System Maintenance Plan of NeighborHelp is guided by the ISO/IEC 25010 software quality model, which defines key quality characteristics that a software product must meet to remain effective, reliable, and maintainable. The plan focuses on sustaining the functional requirements of the system, including user registration and authentication, service discovery and GPS-based matching, booking and scheduling, in-app messaging, rating and review management, analytics reporting, push notifications, audit logging, and data management.

Functional suitability is sustained through regular verification of the system's core modules to confirm that they continue to perform their intended tasks accurately. Usability is maintained through periodic feedback collection from

customers, service providers, and administrators, leading to improvements in interface design and workflow clarity. Reliability is supported by monitoring Firebase services, reviewing backup configurations, preparing rollback procedures, and handling external service issues such as Google Maps Platform or Firebase Cloud Messaging interruptions.

Security is reinforced through periodic review of Firebase Authentication settings, Cloud Firestore security rules, Firebase Storage rules, password policies, session controls, data retention practices, and access permissions. Maintainability is supported by the modular structure of the Flutter application and Firebase services, allowing individual components such as GPS matching, messaging, notifications, reviews, and analytics to be updated without disrupting the entire platform. Portability is maintained by ensuring that the Flutter mobile application and Flutter Web dashboard remain compatible with supported devices, browsers, and Firebase configurations.

Table 15. System Maintenance Plan of NeighborHelp based on ISO/IEC 25010

ISO/IEC 25010 Attribute	Maintenance Activity	Description	Frequency
------------------------------------	---------------------------------	--------------------	------------------

Functional Suitability	Functional Verification	Review all core modules, including user authentication, GPS-based matching, booking management, messaging, rating and review, analytics, notifications, audit trail, and data export to confirm that they continue to perform their intended tasks accurately. Reported functional issues are logged and corrected.	Quarterly
Usability	User Feedback and UI Refinement	Gather feedback from customers, service providers, and administrators regarding interface clarity, navigation ease, form usability, and overall experience on the Flutter mobile application and Flutter Web dashboard. Apply layout and workflow improvements based on feedback.	Every 6 months

Security	Security Rules and Access Review	Review Firebase Authentication settings, Cloud Firestore security rules, Firebase Storage rules, user roles, password policy, session time-out settings, and access permissions. Apply updates to Flutter and Firebase packages when security improvements are released.	Monthly review; updates as needed
Maintainability	Codebase Upkeep and Modular Updates	Maintain the modular structure of the Flutter application and Flutter Web dashboard so that components such as authentication, GPS matching, booking, messaging, notifications, reviews, and analytics can be updated independently. Keep documentation, version control logs, and change records updated.	Ongoing; reviewed each sprint or update cycle

Portability	Deployment and Compatibility Checks	Verify that the Flutter mobile application operates correctly on supported	Every 6 months; before major releases
		Android devices and that the Flutter Web administrative dashboard works properly across major web browsers. Review Firebase configuration to support future scaling and deployment to additional communities.	
Compliance	Data Retention and Privacy Review	Review stored records, archived data, audit logs, and privacyrelated procedures to ensure that personal and location-based data are retained only for legitimate purposes.	Every 6 months

References

- [1] A. Biørn-Hansen, T. A. Majchrzak, and T.-M. Grønli, “Progressive Web Apps for the Unified Development of Mobile Applications,” *Lecture Notes in Business Information Processing*, vol. 322, pp. 64–86, 2018, doi: 10.1007/978-3-319-935270_4.
- [2] A. Biørn-Hansen et al., “An Empirical Investigation of Performance Overhead in Cross-Platform Mobile Development Frameworks,” *Empirical Software Engineering*, vol. 25, pp. 2997–3040, 2020, doi: 10.1007/s10664-020-09827-6.
- [3] A. Biørn-Hansen, T.-M. Grønli, and G. Ghinea, “Animations in Cross-Platform Mobile Applications: An Evaluation of Tools, Metrics and Performance,” *Sensors*, vol. 19, no. 9, 2019, doi: 10.3390/s19092081.
- [4] Organisation for Economic Co-operation and Development (OECD), “Digital

Platforms and the Future of Work,” *OECD Digital Economy Papers*, no. 292, 2019.

[5] Department of Information and Communications Technology (DICT), *Philippine Digital Strategy 2011–2016*, Manila, Philippines, 2011.

[6] Department of Information and Communications Technology (DICT), *National ICT Ecosystem Framework (NICTEF)*, Manila, Philippines, 2019.

[7] Philippine Statistics Authority (PSA), “2024 Functional Literacy, Education and Mass Media Survey (FLEMMS): Internet Usage and Mobile Phone Ownership in Davao Region,” 2024.

[8] J. Abangan, “DICT to install more free WiFi in Davao del Norte,” *Philippine Information Agency*, Aug. 27, 2024.

[9] OECD, *Digital Platforms and the Future of Work*, OECD Publishing, 2019.

[10] S. R. Barley and G. Kunda, “Contracting: A new form of professional practice,” *Academy of Management Perspectives*, vol. 30, no. 4, pp. 356–373, 2016.

[11] A. Sundararajan, *The Sharing Economy: The End of Employment and the Rise of Crowd-Based Capitalism*. Cambridge, MA, USA: MIT Press, 2016.

[12] World Bank, *Philippines Digital Economy Report 2020: A Better Normal Under COVID-19*, Washington, DC, USA, 2020.

[13] J. Hamari, M. Sjöklint, and A. Ukkonen, “The sharing economy: Why people participate in collaborative consumption,” *Journal of the Association for Information Science and Technology*, vol. 67, no. 9, pp. 2047–2059, 2016, doi: 10.1002/asi.23552.

[14] K. Schwaber and J. Sutherland, *The Scrum Guide*, 2020.

- [15] World Bank, *World Development Report 2019: The Changing Nature of Work*, Washington, DC, USA, 2019.
- [16] United Nations, *Transforming Our World: The 2030 Agenda for Sustainable Development*, 2015.
- [17] Davao del Norte State College, “Home,” *Davao del Norte State College*. [Online]. Available: <https://dnsc.edu.ph/>.
- [18] Commission on Higher Education (CHED), *A.C.H.I.E.V.E. Agenda 2030*. Quezon City, Philippines: Commission on Higher Education, 2025. [Online].
- [19] Department of Science and Technology (DOST), *Harmonized National Research and Development Agenda (HNRDA) 2022–2028*. Taguig City, Philippines: Department of Science and Technology, Jul. 2022.
- [20] X. He et al., “Neural Collaborative Filtering,” in *Proc. 26th Int. Conf. World Wide Web (WWW)*, 2017, pp. 173–182, doi: 10.1145/3038912.3052569.
- [21] G. G. Parker, M. W. Van Alstyne, and S. P. Choudary, “Platform ecosystems: How developers invert the firm,” *MIS Quarterly*, vol. 41, no. 1, pp. 255–266, 2017, doi: 10.25300/MISQ/2017/41.1.13.
- [22] G. Adomavicius and A. Tuzhilin, “Toward the next generation of recommender systems: A survey of the state-of-the-art and possible extensions,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 17, no. 6, pp. 734–749, 2005.
- [23] Q. Han and G. Wang, “Design and evaluation of a location-based service system,” *IEEE Transactions on Mobile Computing*, vol. 5, no. 10, pp. 1454–1468, 2006.
- [24] S. Steiniger and A. J. S. Hunter, “A survey of mobile location-based service systems,” *Journal of Location Based Services*, vol. 6, no. 1, pp. 2–26, 2012.

- [25] M. F. Mokbel, "Efficient processing of location-based queries in mobile environments," *Proc. VLDB Endowment*, vol. 2, no. 1, pp. 1001–1012, 2009.
- [26] H. Xiong and P. S. Yu, "Big data analytics in recommender systems," *IEEE Transactions on Knowledge and Data Engineering*, vol. 29, no. 9, pp. 1749–1762, 2017.
- [27] C. Chen et al., "Data analytics for service systems: A review," *Information & Management*, vol. 55, no. 6, pp. 739–755, 2018, doi: 10.1016/j.im.2018.01.005.
- [28] A. Fox and D. Patterson, "Engineering software as a service: An agile approach using cloud computing," *Communications of the ACM*, vol. 53, no. 4, pp. 100–109, 2010.
- [29] R. Kumar and S. Gupta, "A study on service discovery challenges in online and offline environments," *International Journal of Information Systems*, vol. 45, pp. 102–110, 2021.
- [30] G. Karthick and G. Mapp, "Developing a secure system for smart cities," *Future Internet*, vol. 16, no. 9, 2024. <https://www.mdpi.com/1999-5903/16/9/316>
- [31] J. M. Leimeister, S. Krcmar, et al., "Service systems engineering: An interdisciplinary approach to service innovation," *Electronic Markets*, vol. 25, no. 1, pp. 55–66, 2015.
- [32] I. Sommerville, *Software Engineering*, 10th ed. Boston, MA, USA: Pearson, 2016.
- [33] K. Beck et al., "Manifesto for Agile Software Development," 2001.
- [34] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 9th ed. New York, NY, USA: McGraw-Hill Education, 2019. [34] H.

Kerzner, *Project Management: A Systems Approach to Planning, Scheduling, and Controlling*, 12th ed. Hoboken, NJ, USA: Wiley, 2017.

[35] Project Management Institute, *A Guide to the Project Management Body of Knowledge (PMBOK® Guide)*, 6th ed. Newtown Square, PA, USA: Project Management Institute, 2017.

[36] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Boston, MA, USA: Pearson, 2016.

[37] M. Fowler, *Refactoring: Improving the Design of Existing Code*, 2nd ed. Boston, MA, USA: Addison-Wesley, 2018.

[38] Flutter, “Flutter: Build apps for any screen,” 2026. [Online]. Available: <https://flutter.dev>

[39] Dart, “Dart documentation,” 2026. [Online]. Available: <https://dart.dev/docs>

[40] Firebase, “Firebase Authentication documentation,” 2026. [Online]. Available: <https://firebase.google.com/docs/auth>

[41] Firebase, “Cloud Firestore documentation,” 2026. [Online]. Available: <https://firebase.google.com/docs/firestore>

[42] Firebase, “Cloud Storage for Firebase documentation,” 2026. [Online]. Available: <https://firebase.google.com/docs/storage>

[43] Firebase, “Firebase Cloud Messaging documentation,” 2026. [Online]. Available: <https://firebase.google.com/docs/cloud-messaging>

[44] Firebase, “Cloud Functions for Firebase documentation,” 2026. [Online]. Available: <https://firebase.google.com/docs/functions>

[45] Google Maps Platform, “Google Maps Platform documentation,” 2026. [Online]. Available: <https://developers.google.com/maps/documentation>

[46] Flutter, “google_maps_flutter package,” *pub.dev*, 2026. [Online]. Available: https://pub.dev/packages/google_maps_flutter

[47] fl_chart, “fl_chart Flutter package,” *pub.dev*, 2026. [Online]. Available: https://pub.dev/packages/fl_chart

[48] ISO/IEC, *ISO/IEC 27001:2022 Information Security, Cybersecurity and Privacy Protection — Information Security Management Systems —*

Requirements. Geneva, Switzerland: International Organization for Standardization, 2022.

[49] ISO/IEC, *ISO/IEC 25010:2011 Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — System and Software Quality Models*. Geneva, Switzerland: International Organization for Standardization, 2011.

[50] Republic of the Philippines, *Republic Act No. 10173: Data Privacy Act of 2012*. Manila, Philippines: Congress of the Philippines, 2012. [Online]. Available: <https://privacy.gov.ph/data-privacy-act/>

[51] F. Ricci, L. Rokach, and B. Shapira, Eds., *Recommender Systems Handbook*, 3rd ed. New York, NY, USA: Springer, 2022.

[52] C. C. Aggarwal, *Recommender Systems: The Textbook*. Cham, Switzerland: Springer, 2016.

[53] C. Veness, "Calculate distance, bearing and more between latitude/longitude points," *Movable Type Scripts*, 2024. [Online]. Available: <https://www.movabletype.co.uk/scripts/latlong.html>

[54] J. M. Malczewski, *GIS and Multicriteria Decision Analysis*. New York, NY, USA: John Wiley & Sons, 1999.

[55] R. T. Fielding, *Architectural Styles and the Design of Network-based Software Architectures*, Ph.D. dissertation, University of California, Irvine, CA, USA, 2000.

[56] X. He, L. Liao, H. Zhang, L. Nie, X. Hu, and T.-S. Chua, "Neural Collaborative Filtering," in *Proc. 26th Int. Conf. World Wide Web (WWW)*, Perth, Australia, 2017, pp. 173–182, doi: 10.1145/3038912.3052569.

[57] L. Shi and S. Viswanathan, "Optional Verification and Signaling in Online Matching Markets: Evidence from a Randomized Field Experiment," *Information Systems Research*, vol. 34, no. 4, pp. 1603–1621, Dec. 2023, doi: 10.1287/isre.2022.1194.

[58] Y. Jang and S. Kim, "The Factors Influencing Users' Trust in and Loyalty to Consumer-to-Consumer Secondhand Marketplace Platform," *Behavioral Sciences*, vol. 13, no. 3, Art. no. 242, 2023, doi: 10.3390/bs13030242.

[59] [32] R. Latif, B. M. Yakubu, and T. Saba, "MarketTrust: Blockchain-Based Trust Evaluation Model for SIoT-Based Smart Marketplaces," *Scientific Reports*, vol. 13, Art. no. 11571, 2023, doi: 10.1038/s41598-023-38078-w.